A cross-package Bioconductor workflow for analysing methylation array data

Jovana Maksimovic*[*]*, Belinda Phipson and Alicia Oshlack

[*]jovana.maksimovic@mcri.edu.au (mailto:jovana.maksimovic@mcri.edu.au)

30 April 2024

Abstract

Methylation in the human genome is known to be associated with development and disease. The Illumina Infinium methylation arrays are by far the most common way to interrogate methylation across the human genome. This paper provides a Bioconductor workflow using multiple packages for the analysis of methylation array data. Specifically, we demonstrate the steps involved in a typical differential methylation analysis pipeline including: quality control, filtering, normalization, data exploration and statistical testing for probe-wise differential methylation. We further outline other analyses such as differential methylation of regions, differential variability analysis, estimating cell type composition and gene ontology testing. Finally, we provide some examples of how to visualise methylation array data.

Contents

1 Introduction

2 Differential methylation analysis

- 2.1 Obtaining the data
- 2.2 Loading the data
- 2.3 Quality control
- 2.4 Normalisation
- 2.5 Data exploration
- 2.6 Filtering
- 2.7 Probe-wise differential methylation analysis
- 2.8 Differential methylation analysis of regions
- 2.9 Customising visualisations of methylation data

3 Additional analyses

- 3.1 Gene ontology testing
- 3.2 Differential variability
- 3.3 Cell type composition

4 Discussion

5 Software versions

6 Author contributions

7 Competing interests

8 Grant information

References

R version: R version 4.4.0 RC (2024-04-16 r86468)

Bioconductor version: 3.19

Package: 1.29.0

1 Introduction

DNA methylation, the addition of a methyl group to a CG dinucleotide of the DNA, is the most extensively studied epigenetic mark due to its role in both development and disease (Bird 2002; Laird 2003). Although DNA methylation can be measured in several ways, the epigenetics community has enthusiastically embraced the Illumina HumanMethylation450 (450k) array (Bibikova et al. 2011) as a cost-effective way to assay methylation across the human genome. More recently, Illumina has increased the genomic coverage of the platform to >850,000 sites with the release of their MethylationEPIC (850k) array. As methylation arrays are likely to remain popular for measuring methylation for the foreseeable future, it is necessary to provide robust workflows for methylation array analysis.

Measurement of DNA methylation by Infinium technology (Infinium I) was first employed by Illumina on the HumanMethylation27 (27k) array (Bibikova et al. 2009), which measured methylation at approximately 27,000 CpGs, primarily in gene promoters. Like bisulfite sequencing, the Infinium assay detects methylation status at single base resolution. However, due to its relatively limited coverage the array platform was not truly considered “genome-wide” until the arrival of the 450k array. The 450k array increased the genomic coverage of the platform to over 450,000 gene-centric sites by combining the original Infinium I assay with the novel Infinium II probes. Both assay types employ 50bp probes that query a [C/T] polymorphism created by bisulfite conversion of unmethylated cytosines in the genome, however, the Infinium I and II assays differ in the number of beads required to detect methylation at a single locus. Infinium I uses two bead types per CpG, one for each of the methylated and unmethylated states (Figure 1a). In contrast, the Infinium II design uses one bead type and the methylated state is determined at the single base extension step after hybridization (Figure 1b). The 850k array also uses a combination of the Infinium I and II assays but achieves additional coverage by increasing the size of each array; a 450k slide contains 12 arrays whilst the 850k has only 8.

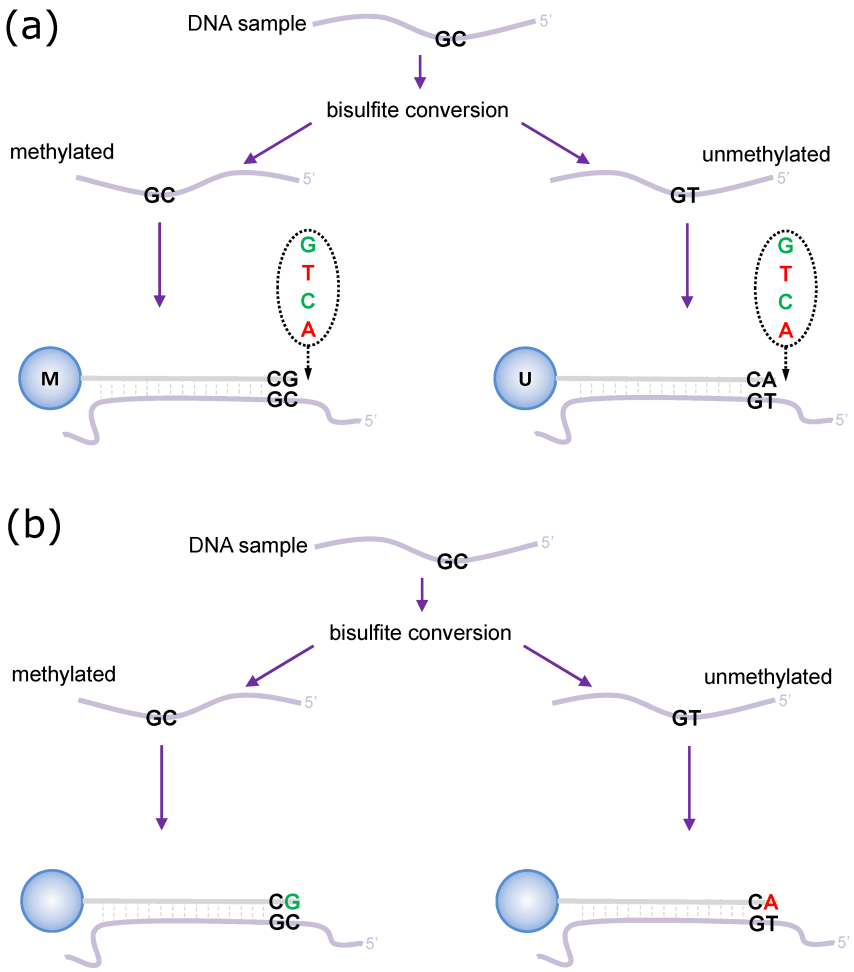


Figure 1: **Illumina Infinium HumanMethylation450 assay, reproduced from Maksimovic, Gordon and Oshlack 2012**
(a) Infinium I assay. Each individual CpG is interrogated using two bead types: methylated (M) and unmethylated (U). Both bead types will incorporate the same labeled nucleotide for the same target CpG, thereby producing the same color fluorescence. The nucleotide that is added is determined by the base downstream of the 'C' of the target CpG. The proportion of methylation can be calculated by comparing the intensities from the two different probes in the same color. (b) Infinium II assay. Each target CpG is interrogated using a single bead type. Methylation state is detected by single base extension at the position of the 'C' of the target CpG, which always results in the addition of a labeled 'G' or 'A' nucleotide, complementary to either the 'methylated' C or 'unmethylated' T, respectively. Each locus is detected in two colors, and methylation status is determined by comparing the two colors from the one position.

Regardless of the Illumina array version, for each CpG, there are two measurements: a methylated intensity (denoted by M) and an unmethylated intensity (denoted by U). These intensity values can be used to determine the proportion of methylation at each CpG locus. Methylation levels are commonly reported as either beta values ($\beta = M/(M+U)$) or M-values ($Mvalue = log2(M/U)$). For practical purposes, a small offset, α, can be added to the denominator of the β value equation to avoid dividing by small values, which is the default behaviour of the `getBeta` function in *minfi*. The default value for α is 100. It may also be desirable to add a small offset to the numerator and denominator when calculating M-values to avoid dividing by zero in rare cases, however the default `getM` function in *minfi* does not do this. Beta values and M-values are related through a logit transformation. Beta values are generally preferable for describing the level of methylation at a locus or for graphical presentation because percentage methylation is easily interpretable. However, due to their distributional properties, M-values are more appropriate for statistical testing (Du et al. 2010).

In this workflow, we will provide examples of the steps involved in analysing methylation array data using R (R Core Team 2014) and Bioconductor (Huber et al. 2015), including: quality control, filtering, normalization, data exploration and probe-wise differential methylation analysis. We will also cover other approaches such as differential methylation analysis of regions, differential variability analysis, gene ontology analysis and estimating cell type composition. Finally, we will provide some examples of useful ways to visualise methylation array data.

2 Differential methylation analysis

2.1 Obtaining the data

The data required for this workflow has been bundled with the R package that contains this workflow document. Alternatively, it can be obtained from figshare (https://figshare.com/s/7a37f43c0ca2fec4669e). If you choose to download it seperately, once the data has been downloaded, it needs to be extracted from the archive. This will create a folder called `data`, which contains all the files necessary to execute the workflow.

Once the data has been downloaded and extracted, there should be a folder called `data` that contains all the files necessary to execute the workflow.

```
# set up a path to the data directory
dataDirectory <- system.file("extdata", package = "methylationArrayAr
# list the files
list.files(dataDirectory, recursive = TRUE)
```

```
##  [1] "48639-non-specific-probes-Illumina450k.csv"
##  [2] "5975827018/5975827018_R06C02_Grn.idat"
##  [3] "5975827018/5975827018_R06C02_Red.idat"
##  [4] "6264509100/6264509100_R01C01_Grn.idat"
##  [5] "6264509100/6264509100_R01C01_Red.idat"
##  [6] "6264509100/6264509100_R01C02_Grn.idat"
##  [7] "6264509100/6264509100_R01C02_Red.idat"
##  [8] "6264509100/6264509100_R02C01_Grn.idat"
##  [9] "6264509100/6264509100_R02C01_Red.idat"
## [10] "6264509100/6264509100_R02C02_Grn.idat"
## [11] "6264509100/6264509100_R02C02_Red.idat"
## [12] "6264509100/6264509100_R03C01_Grn.idat"
## [13] "6264509100/6264509100_R03C01_Red.idat"
## [14] "6264509100/6264509100_R03C02_Grn.idat"
## [15] "6264509100/6264509100_R03C02_Red.idat"
## [16] "6264509100/6264509100_R04C01_Grn.idat"
## [17] "6264509100/6264509100_R04C01_Red.idat"
## [18] "6264509100/6264509100_R04C02_Grn.idat"
## [19] "6264509100/6264509100_R04C02_Red.idat"
## [20] "6264509100/6264509100_R05C01_Grn.idat"
## [21] "6264509100/6264509100_R05C01_Red.idat"
## [22] "6264509100/6264509100_R05C02_Grn.idat"
## [23] "6264509100/6264509100_R05C02_Red.idat"
## [24] "6264509100/6264509100_R06C01_Grn.idat"
## [25] "6264509100/6264509100_R06C01_Red.idat"
## [26] "6264509100/6264509100_R06C02_Grn.idat"
## [27] "6264509100/6264509100_R06C02_Red.idat"
## [28] "SampleSheet.csv"
## [29] "ageData.RData"
## [30] "human_c2_v5.rdata"
## [31] "model-based-cpg-islands-hg19-chr17.txt"
## [32] "wgEncodeRegDnaseClusteredV3chr17.bed"
```

To demonstrate the various aspects of analysing methylation data, we will be using a small, publicly available 450k methylation dataset (GSE49667 (http://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE49667))(Zhang et al. 2013). The dataset contains 10 samples in total: there are 4 different sorted T-cell types (naive, rTreg, act_naive, act_rTreg, collected from 3 different individuals (M28, M29, M30). For details describing sample collection and preparation, see Zhang et al. (2013). An additional `birth` sample (individual VICS-72098-18-B) is included from another study (GSE51180 (http://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE51180))(Cruickshank et al. 2013) to illustrate approaches for identifying and excluding poor quality samples.

There are several R Bioconductor packages available that have been developed for analysing methylation array data, including *minfi* (Aryee et al. 2014), *missMethyl* (Phipson, Maksimovic, and Oshlack 2016), *wateRmelon* (Pidsley et al. 2013), *methylumi* (Davis et al. 2015), *ChAMP* (Morris et al. 2014) and *charm* (Aryee et al. 2011). Some of the packages, such as *minfi* and *methylumi* include a framework for reading in the raw data from IDAT files and various specialised objects for storing and manipulating the data throughout the course of an analysis. Other packages provide specialised analysis methods for normalisation and statistical testing that rely on either *minfi* or *methylumi* objects. It is possible to convert between *minfi* and *methylumi* data types, however, this is not always trivial. Thus, it is advisable to

consider the methods that you are interested in using and the data types that are most appropriate before you begin your analysis. Another popular method for analysing methylation array data is *limma* (Ritchie et al. 2015), which was originally developed for gene expression microarray analysis. As *limma* operates on a matrix of values, it is easily applied to any data that can be converted to a `matrix` in R. For a complete list of Bioconductor packages for analysing DNA methylation data, one can search for "DNAMethylation" in BiocViews (https://www.bioconductor.org/packages/release/BiocViews.html#___DNAMethylation (https://www.bioconductor.org/packages/release/BiocViews.html#___DNAMethylation)) on the Bioconductor website (https://www.bioconductor.org/).

We will begin with an example of a **probe-wise** differential methylation analysis using *minfi* and *limma*. By **probe-wise** analysis we mean each individual CpG probe will be tested for differential methylation for the comparisons of interest and p-values and moderated t-statistics (Smyth 2004) will be generated for each CpG probe.

2.2 Loading the data

It is useful to begin an analysis in R by loading all the packages that are likely to be required.

```
# load packages required for analysis
library(knitr)
library(limma)
library(minfi)
library(IlluminaHumanMethylation450kanno.ilmn12.hg19)
library(IlluminaHumanMethylation450kmanifest)
library(RColorBrewer)
library(missMethyl)
library(minfiData)
library(Gviz)
library(DMRcate)
library(stringr)
```

The *minfi*, *IlluminaHumanMethylation450kanno.ilmn12.hg19*, *IlluminaHumanMethylation450kmanifest*, *missMethyl*, *minfiData* and *DMRcate* are methylation specific packages, while *RColorBrewer* and *Gviz* are visualisation packages. We use *limma* for testing differential methylation, and *matrixStats* and *stringr* have functions used in the workflow. The *IlluminaHumanMethylation450kmanifest* package provides the Illumina manifest as an R object which can easily be loaded into the environment. The manifest contains all of the annotation information for each of the CpG probes on the 450k array. This is useful for determining where any differentially methylated probes are located in a genomic context.

```
# get the 450k annotation data
ann450k <- getAnnotation(IlluminaHumanMethylation450kanno.ilmn12.hg19
head(ann450k)
```

```
## DataFrame with 6 rows and 33 columns
##                    chr       pos      strand        Name    Addres
##            <character> <integer> <character> <character> <characte
## cg00050873       chrY   9363356           -  cg00050873    327353
## cg00212031       chrY  21239348           -  cg00212031    296744
## cg00213748       chrY   8148233           -  cg00213748    307034
## cg00214611       chrY  15815688           -  cg00214611    697923
## cg00455876       chrY   9385539           -  cg00455876    276534
## cg01707559       chrY   6778695           +  cg01707559    456524
##               AddressB             ProbeSeqA             ProbeSe
##            <character>           <character>           <characte
## cg00050873    31717405 ACAAAAAAACAACACACAAC.. ACGAAAAAACAACGCACAAC
## cg00212031    38703326 CCCAATTAACCACAAAAACT.. CCCAATTAACCGCAAAAACT
## cg00213748    36767301 TTTTAACACCTAACACCATT.. TTTTAACGCCTAACACCGTT
## cg00214611    46723459 CTAACTTCCAAACCACACTT.. CTAACTTCCGAACCGCGCTT
## cg00455876    69732350 AACTCTAAACTACCCAACAC.. AACTCTAAACTACCCGACAC
## cg01707559    64689504 ACAAATTAAAAACACTAAAA.. GCGAATTAAAAACACTAAAA
##                   Type    NextBase       Color    Probe_rs Probe_m
##            <character> <character> <character> <character> <numeri
## cg00050873          I           A         Red          NA
## cg00212031          I           T         Red          NA
## cg00213748          I           A         Red          NA
## cg00214611          I           A         Red          NA
## cg00455876          I           A         Red          NA
## cg01707559          I           A         Red          NA
##                CpG_rs   CpG_maf      SBE_rs   SBE_maf           I
##            <character> <numeric> <character> <numeric>
## cg00050873         NA        NA          NA        NA   chrY:9363
## cg00212031         NA        NA          NA        NA chrY:212384
## cg00213748         NA        NA          NA        NA   chrY:8147
## cg00214611         NA        NA          NA        NA chrY:158154
## cg00455876         NA        NA          NA        NA   chrY:9385
## cg01707559         NA        NA          NA        NA   chrY:6778
##            Relation_to_Island       Forward_Sequence
##                   <character>           <character>           <c
## cg00050873           N_Shore TATCTCTGTCTGGCGAGGAG.. CGGGGTCCACCCA
## cg00212031            Island CCATTGGCCCGCCCCAGTTG.. CGCACGTCTTCCC
## cg00213748           S_Shore TCTGTGGGACCATTTTAACG.. CGCCCCCTCCTGC
## cg00214611            Island GCGCCGGCAGGACTAGCTTC.. CGCCCGCGCCACA
## cg00455876            Island CGCGTGTGCCTGGACTCTGA.. GACTCTGAGCTAC
## cg01707559            Island AGCGGCCGCTCCCAGTGGTG.. CGCCCTCTGTCGC
##            Random_Loci Methyl27_Loci UCSC_RefGene_Name UCSC_RefGer
##            <character>   <character>       <character>
## cg00050873                             TSPY4;FAM197Y2 NM_00116447
## cg00212031                                    TTTY14
## cg00213748
## cg00214611                              TMSB4Y;TMSB4Y    NM_00420
## cg00455876
## cg01707559                            TBL1Y;TBL1Y;TBL1Y NM_134259;N
##              UCSC_RefGene_Group     Phantom         DMR    Enhance
##                     <character> <character> <character> <character
## cg00050873        Body;TSS1500
## cg00212031             TSS200
## cg00213748
## cg00214611       1stExon;5'UTR
## cg00455876
```

```
## cg01707559 TSS200;TSS200;TSS200
##                     HMM_Island Regulatory_Feature_Name Regulatory_
##                    <character>             <character>
## cg00050873   Y:9973136-9976273
## cg00212031 Y:19697854-19699393
## cg00213748   Y:8207555-8208234
## cg00214611 Y:14324883-14325218     Y:15815422-15815706   Promoter_
## cg00455876   Y:9993394-9995882
## cg01707559   Y:6838022-6839951
##                          DHS
##                  <character>
## cg00050873
## cg00212031
## cg00213748
## cg00214611
## cg00455876
## cg01707559
```

As for their many other BeadArray platforms, Illumina methylation data is usually obtained in the form of Intensity Data (IDAT) Files. This is a proprietary format that is output by the scanner and stores summary intensities for each probe on the array. However, there are Bioconductor packages available that facilitate the import of data from IDAT files into R (Smith et al. 2013). Typically, each IDAT file is approximately 8MB in size. The simplest way to import the raw methylation data into R is using the *minfi* function `read.metharray.sheet`, along with the path to the IDAT files and a sample sheet. The sample sheet is a CSV (comma-separated) file containing one line per sample, with a number of columns describing each sample. The format expected by the `read.metharray.sheet` function is based on the sample sheet file that usually accompanies Illumina methylation array data. It is also very similar to the targets file described by the *limma* package. Importing the sample sheet into R creates a `data.frame` with one row for each sample and several columns. The `read.metharray.sheet` function uses the specified path and other information from the sample sheet to create a column called `Basename` which specifies the location of each individual IDAT file in the experiment.

```
# read in the sample sheet for the experiment
targets <- read.metharray.sheet(dataDirectory, pattern="SampleSheet.c
targets
```

Now that we have imported the information about the samples and where the data is located, we can read the raw intensity signals into R from the IDAT files using the `read.metharray.exp` function. This creates an `RGChannelSet` object that contains all the raw intensity data, from both the red and green colour channels, for each of the samples. At this stage, it can be useful to rename the samples with more descriptive names.

```
# read in the raw data from the IDAT files
rgSet <- read.metharray.exp(targets=targets)
rgSet
```

```
## class: RGChannelSet
## dim: 622399 11
## metadata(0):
## assays(2): Green Red
## rownames(622399): 10600313 10600322 ... 74810490 74810492
## rowData names(0):
## colnames(11): 6264509100_R01C01 6264509100_R02C01 ... 6264509100_R
##   5975827018_R06C02
## colData names(10): Sample_Name Sample_Well ... Basename filenames
## Annotation
##   array: IlluminaHumanMethylation450k
##   annotation: ilmn12.hg19
```

```
# give the samples descriptive names
targets$ID <- paste(targets$Sample_Group,targets$Sample_Name,sep=".")
sampleNames(rgSet) <- targets$ID
rgSet
```

```
## class: RGChannelSet
## dim: 622399 11
## metadata(0):
## assays(2): Green Red
## rownames(622399): 10600313 10600322 ... 74810490 74810492
## rowData names(0):
## colnames(11): naive.1 rTreg.2 ... act_rTreg.10 birth.11
## colData names(10): Sample_Name Sample_Well ... Basename filenames
## Annotation
##   array: IlluminaHumanMethylation450k
##   annotation: ilmn12.hg19
```

2.3 Quality control

Once the data has been imported into R, we can evaluate its quality. Firstly, we need to calculate detection p-values. We can generate a detection p-value for every CpG in every sample, which is indicative of the quality of the signal. The method used by *minfi* to calculate detection p-values compares the total signal $(M + U)$ for each probe to the background signal level, which is estimated from the negative control probes. Very small p-values are indicative of a reliable signal whilst large p-values, for example >0.01, generally indicate a poor quality signal.

Plotting the mean detection p-value for each sample allows us to gauge the general quality of the samples in terms of the overall signal reliability (Figure 2). Samples that have many failed probes will have relatively large mean detection p-values.

```
# calculate the detection p-values
detP <- detectionP(rgSet)
head(detP)
```

```
##                 naive.1       rTreg.2  act_naive.3     naive.4
## cg00050873  0.000000e+00  0.000000e+00 0.000000e+00 0.00000e+00  0
## cg00212031  0.000000e+00  0.000000e+00 0.000000e+00 0.00000e+00  0
## cg00213748  2.139652e-88  4.213813e-31 1.181802e-12 1.29802e-47  8
## cg00214611  0.000000e+00  0.000000e+00 0.000000e+00 0.00000e+00  0
## cg00455876 1.400696e-234 9.349236e-111 4.272105e-90 0.00000e+00 3.
## cg01707559  0.000000e+00  0.000000e+00 0.000000e+00 0.00000e+00  0
##              act_rTreg.6      naive.7       rTreg.8  act_naive.9
## cg00050873  0.000000e+00  0.00000e+00  0.000000e+00 0.000000e+00
## cg00212031  0.000000e+00  0.00000e+00  0.000000e+00 0.000000e+00
## cg00213748  2.592206e-23  1.16160e-28  1.469801e-05 1.543654e-21
## cg00214611  0.000000e+00  0.00000e+00  0.000000e+00 0.000000e+00
## cg00455876 4.690740e-308 1.08647e-219 5.362780e-178 0.000000e+00 7
## cg01707559  0.000000e+00  0.00000e+00  0.000000e+00 0.000000e+00
##                birth.11
## cg00050873  0.000000e+00
## cg00212031 2.638199e-237
## cg00213748  6.735224e-01
## cg00214611  7.344451e-01
## cg00455876 4.403634e-174
## cg01707559  0.000000e+00
```

```
# examine mean detection p-values across all samples to identify any
pal <- brewer.pal(8,"Dark2")
par(mfrow=c(1,2))
barplot(colMeans(detP), col=pal[factor(targets$Sample_Group)], las=2,
        cex.names=0.8, ylab="Mean detection p-values")
abline(h=0.05,col="red")
legend("topleft", legend=levels(factor(targets$Sample_Group)), fill=p
       bg="white")

barplot(colMeans(detP), col=pal[factor(targets$Sample_Group)], las=2,
        cex.names=0.8, ylim=c(0,0.002), ylab="Mean detection p-values
abline(h=0.05,col="red")
legend("topleft", legend=levels(factor(targets$Sample_Group)), fill=p
       bg="white")
```

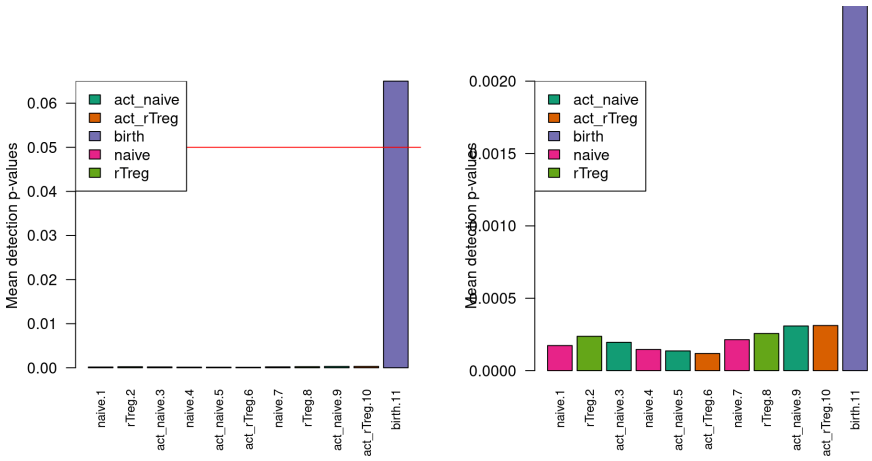


Figure 2: **Mean detection p-values summarise the quality of the signal across all the probes in each sample**
The plot on the right is a zoomed in version of the plot on the left.

The *minfi* `qcReport` function generates many other useful quality control plots. The *minfi* vignette (http://bioconductor.org/packages/release/bioc/vignettes/minfi/inst/doc/minfi.pdf) describes the various plots and how they should be interpreted in detail. Generally, samples that look poor based on mean detection p-value will also look poor using other metrics and it is usually advisable to exclude them from further analysis.

```
qcReport(rgSet, sampNames=targets$ID, sampGroups=targets$Sample_Group
         pdf="qcReport.pdf")
```

Poor quality samples can be easily excluded from the analysis using a detection p-value cutoff, for example >0.05. For this particular dataset, the `birth` sample shows a very high mean detection p-value, and hence it is excluded from subsequent analysis (Figure 2).

```
# remove poor quality samples
keep <- colMeans(detP) < 0.05
rgSet <- rgSet[,keep]
rgSet
```

```
## class: RGChannelSet
## dim: 622399 10
## metadata(0):
## assays(2): Green Red
## rownames(622399): 10600313 10600322 ... 74810490 74810492
## rowData names(0):
## colnames(10): naive.1 rTreg.2 ... act_naive.9 act_rTreg.10
## colData names(10): Sample_Name Sample_Well ... Basename filenames
## Annotation
##   array: IlluminaHumanMethylation450k
##   annotation: ilmn12.hg19
```

```
# remove poor quality samples from targets data
targets <- targets[keep,]
targets[,1:5]
```

```
##    Sample_Name Sample_Well Sample_Source Sample_Group Sample_Label
## 1            1          A1           M28        naive        naive
## 2            2          B1           M28        rTreg        rTreg
## 3            3          C1           M28    act_naive    act_naive
## 4            4          D1           M29        naive        naive
## 5            5          E1           M29    act_naive    act_naive
## 6            6          F1           M29    act_rTreg    act_rTreg
## 7            7          G1           M30        naive        naive
## 8            8          H1           M30        rTreg        rTreg
## 9            9          A2           M30    act_naive    act_naive
## 10          10          B2           M30    act_rTreg    act_rTreg
```

```
# remove poor quality samples from detection p-value table
detP <- detP[,keep]
dim(detP)
```

```
## [1] 485512     10
```

2.4 Normalisation

To minimise the unwanted variation within and between samples, various data normalisations can be applied. Many different types of normalisation have been developed for methylation arrays and it is beyond the scope of this workflow to compare and contrast all of them (Fortin et al. 2014; Wu et al. 2014; Sun et al. 2011; Wang et al. 2012; Maksimovic, Gordon, and Oshlack 2012; Mancuso et al. 2011; Touleimat and Tost 2012; Teschendorff et al. 2013; Pidsley et al. 2013; Triche et al. 2013). Several methods have been built into *minfi* and can be directly applied within its framework (Fortin et al. 2014; Triche et al. 2013; Maksimovic, Gordon, and Oshlack 2012; Touleimat and Tost 2012), whilst others are *methylumi*-specific or require custom data types (Wu et al. 2014; Sun et al. 2011; Wang et al. 2012; Mancuso et al. 2011; Teschendorff et al. 2013; Pidsley et al. 2013). Although there is no single normalisation method that is universally considered best, a recent study by Fortin et al. (2014) has suggested that a good rule of thumb within the *minfi* framework is that the `preprocessFunnorm` (Fortin et al. 2014) function is most appropriate for datasets with global methylation differences such as cancer/normal or vastly different tissue types, whilst the `preprocessQuantile` function (Touleimat and Tost 2012) is more suited for datasets where you do not expect global differences between your samples, for example a single tissue. Further discussion on appropriate choice of normalisation can be found in (Hicks and Irizarry 2015), and the accompanying *quantro* package includes data-driven tests for the assumptions of quantile normalisation. As we are comparing different blood cell types, which are globally relatively similar, we will apply the `preprocessQuantile` method to our data (Figure 3). This function implements a stratified quantile normalisation procedure which is applied to the methylated and unmethylated signal intensities separately, and takes into account the different probe types. Note that after normalisation, the data is housed in a `GenomicRatioSet` object. This is a much more compact representation of the data as the colour channel information has been discarded and the M and U intensity information has been converted to M-values and beta values, together with associated genomic coordinates. Note, running the `preprocessQuantile` function on this dataset produces the warning: **'An inconsistency was encountered while determining sex'**; this can be ignored as it is due to all the samples being from male donors.

```
# normalize the data; this results in a GenomicRatioSet object
mSetSq <- preprocessQuantile(rgSet)
```

```
## [preprocessQuantile] Mapping to genome.
```

```
## Warning in .getSex(CN = CN, xIndex = xIndex, yIndex = yIndex, cutc
## An inconsistency was encountered while determining sex. One possib
## only one sex is present. We recommend further checks, for example
## plotSex function.
```

```
## [preprocessQuantile] Fixing outliers.
```

```
## [preprocessQuantile] Quantile normalizing.
```

```
# create a MethylSet object from the raw data for plotting
mSetRaw <- preprocessRaw(rgSet)
```

```
# visualise what the data looks like before and after normalisation
par(mfrow=c(1,2))
densityPlot(rgSet, sampGroups=targets$Sample_Group,main="Raw", legenc
legend("top", legend = levels(factor(targets$Sample_Group)),
       text.col=brewer.pal(8,"Dark2"))
densityPlot(getBeta(mSetSq), sampGroups=targets$Sample_Group,
            main="Normalized", legend=FALSE)
legend("top", legend = levels(factor(targets$Sample_Group)),
       text.col=brewer.pal(8,"Dark2"))
```

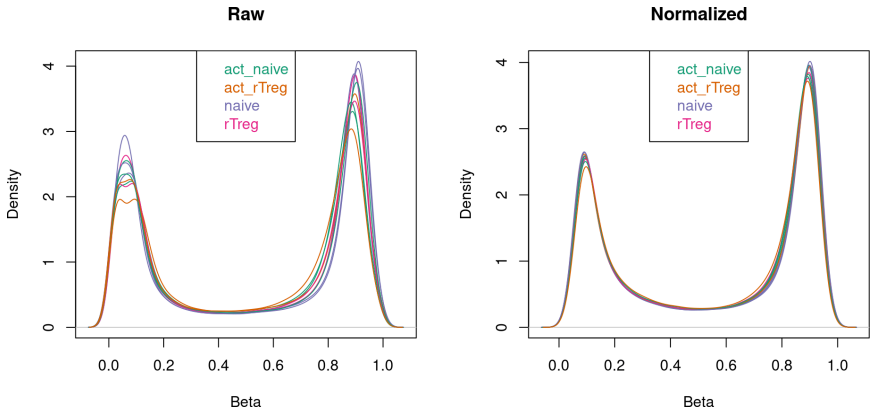


Figure 3: **The density plots show the distribution of the beta values for each sample before and after normalisation**

2.5 Data exploration

Multi-dimensional scaling (MDS) plots are excellent for visualising data, and are usually some of the first plots that should be made when exploring the data. MDS plots are based on principal components analysis and are an unsupervised method for looking at the similarities and differences between the various samples. Samples that are more similar to each other should cluster together, and samples that are very different should be further apart on the plot. Dimension one (or principal component one) captures the greatest source of variation in the data, dimension two captures the second greatest source of variation in the data and so on. Colouring the data points or labels by known factors of interest can often highlight exactly what the greatest sources of variation are in the data. It is also possible to use MDS plots to decipher sample mix-ups.

```
# MDS plots to look at largest sources of variation
par(mfrow=c(1,2))
plotMDS(getM(mSetSq), top=1000, gene.selection="common",
        col=pal[factor(targets$Sample_Group)])
legend("top", legend=levels(factor(targets$Sample_Group)), text.col=p
       bg="white", cex=0.7)

plotMDS(getM(mSetSq), top=1000, gene.selection="common",
        col=pal[factor(targets$Sample_Source)])
legend("top", legend=levels(factor(targets$Sample_Source)), text.col=
       bg="white", cex=0.7)
```

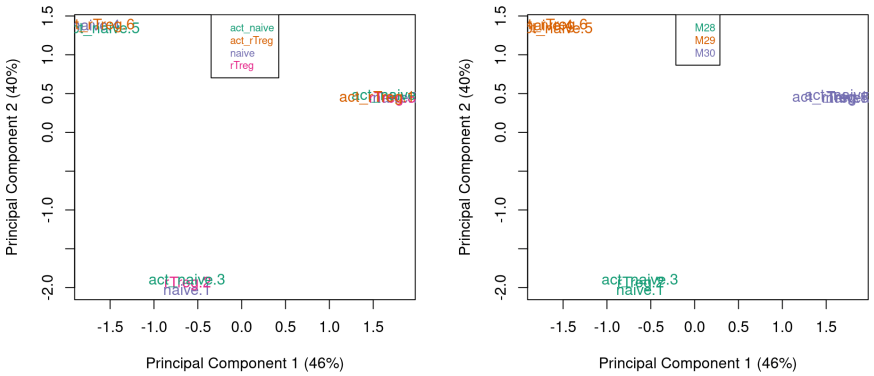


Figure 4: **Multi-dimensional scaling plots are a good way to visualise the relationships between the samples in an experiment**

Examining the MDS plots for this dataset demonstrates that the largest source of variation is the difference between individuals (Figure 4). The higher dimensions reveal that the differences between cell types are largely captured by the third and fourth principal components (Figure 5). This type of information is useful in that it can inform downstream analysis. If obvious sources of unwanted variation are revealed by the MDS plots, we can include them in our statistical model to account for them. In the case of this particular dataset, we will include individual to individual variation in our statistical model.

```
# Examine higher dimensions to look at other sources of variation
par(mfrow=c(1,3))
plotMDS(getM(mSetSq), top=1000, gene.selection="common",
        col=pal[factor(targets$Sample_Group)], dim=c(1,3))
legend("top", legend=levels(factor(targets$Sample_Group)), text.col=p
       cex=0.7, bg="white")

plotMDS(getM(mSetSq), top=1000, gene.selection="common",
        col=pal[factor(targets$Sample_Group)], dim=c(2,3))
legend("topleft", legend=levels(factor(targets$Sample_Group)), text.c
       cex=0.7, bg="white")

plotMDS(getM(mSetSq), top=1000, gene.selection="common",
        col=pal[factor(targets$Sample_Group)], dim=c(3,4))
legend("topright", legend=levels(factor(targets$Sample_Group)), text.
       cex=0.7, bg="white")
```

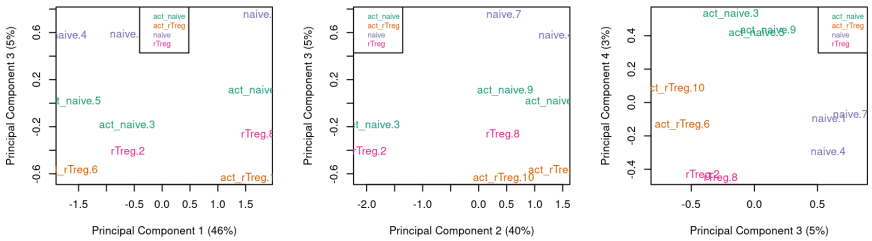


Figure 5: **Examining the higher dimensions of an MDS plot can reaveal significant sources of variation in the data**

2.6 Filtering

Poor performing probes are generally filtered out prior to differential methylation analysis. As the signal from these probes is unreliable, by removing them we perform fewer statistical tests and thus incur a reduced multiple testing penalty. We filter out probes that have failed in one or more samples based on detection p-value.

```
# ensure probes are in the same order in the mSetSq and detP objects
detP <- detP[match(featureNames(mSetSq),rownames(detP)),]

# remove any probes that have failed in one or more samples
keep <- rowSums(detP < 0.01) == ncol(mSetSq)
table(keep)
```

```
## keep
##  FALSE   TRUE
##    977 484535
```

```
mSetSqFlt <- mSetSq[keep,]
mSetSqFlt
```

```
## class: GenomicRatioSet
## dim: 484535 10
## metadata(0):
## assays(2): M CN
## rownames(484535): cg13869341 cg14008030 ... cg08265308 cg14273923
## rowData names(0):
## colnames(10): naive.1 rTreg.2 ... act_naive.9 act_rTreg.10
## colData names(13): Sample_Name Sample_Well ... yMed predictedSex
## Annotation
##   array: IlluminaHumanMethylation450k
##   annotation: ilmn12.hg19
## Preprocessing
##   Method: Raw (no normalization or bg correction)
##   minfi version: 1.49.1
##   Manifest version: 0.4.0
```

Depending on the nature of your samples and your biological question you may also choose to filter out the probes from the X and Y chromosomes or probes that are known to have common SNPs at the CpG site. As the samples in this dataset were all derived from male donors, we will not be removing the sex chromosome probes

as part of this analysis, however example code is provided below. A different dataset, which contains both male and female samples, is used to demonstrate a Differential Variability analysis and provides an example of when sex chromosome removal is necessary (Figure 13).

```
# if your data includes males and females, remove probes on the sex c
keep <- !(featureNames(mSetSqFlt) %in% ann450k$Name[ann450k$chr %in%
                                                    c("chrX","chr
table(keep)
mSetSqFlt <- mSetSqFlt[keep,]
```

There is a function in *minfi* that provides a simple interface for the removal of probes where common SNPs may affect the CpG. You can either remove all probes affected by SNPs (default), or only those with minor allele frequencies greater than a specified value.

```
# remove probes with SNPs at CpG site
mSetSqFlt <- dropLociWithSnps(mSetSqFlt)
mSetSqFlt
```

```
## class: GenomicRatioSet
## dim: 467351 10
## metadata(0):
## assays(2): M CN
## rownames(467351): cg13869341 cg14008030 ... cg08265308 cg14273923
## rowData names(0):
## colnames(10): naive.1 rTreg.2 ... act_naive.9 act_rTreg.10
## colData names(13): Sample_Name Sample_Well ... yMed predictedSex
## Annotation
##   array: IlluminaHumanMethylation450k
##   annotation: ilmn12.hg19
## Preprocessing
##   Method: Raw (no normalization or bg correction)
##   minfi version: 1.49.1
##   Manifest version: 0.4.0
```

We will also filter out probes that have shown to be cross-reactive, that is, probes that have been demonstrated to map to multiple places in the genome. This list was originally published by Chen et al. (2013) and can be obtained from the authors' website (http://www.sickkids.ca/MS-Office-Files/Research/Weksberg%20Lab/48639-non-specific-probes-Illumina450k.xlsx).

```
# exclude cross reactive probes
xReactiveProbes <- read.csv(file=paste(dataDirectory,
                                       "48639-non-specific-probes-Ill
                                       sep="/"), stringsAsFactors=FAL
keep <- !(featureNames(mSetSqFlt) %in% xReactiveProbes$TargetID)
table(keep)
```

```
## keep
##  FALSE   TRUE
##  27433 439918

mSetSqFlt <- mSetSqFlt[keep,]
mSetSqFlt

## class: GenomicRatioSet
## dim: 439918 10
## metadata(0):
## assays(2): M CN
## rownames(439918): cg13869341 cg24669183 ... cg08265308 cg14273923
## rowData names(0):
## colnames(10): naive.1 rTreg.2 ... act_naive.9 act_rTreg.10
## colData names(13): Sample_Name Sample_Well ... yMed predictedSex
## Annotation
##   array: IlluminaHumanMethylation450k
##   annotation: ilmn12.hg19
## Preprocessing
##   Method: Raw (no normalization or bg correction)
##   minfi version: 1.49.1
##   Manifest version: 0.4.0
```

Once the data has been filtered and normalised, it is often useful to re-examine the MDS plots to see if the relationship between the samples has changed. It is apparent from the new MDS plots that much of the inter-individual variation has been removed as this is no longer the first principal component (Figure 6), likely due to the removal of the SNP-affected CpG probes. However, the samples do still cluster by individual in the second dimension (Figure 6 and Figure 7) and thus a factor for individual should still be included in the model.

```
par(mfrow=c(1,2))
plotMDS(getM(mSetSqFlt), top=1000, gene.selection="common",
        col=pal[factor(targets$Sample_Group)], cex=0.8)
legend("right", legend=levels(factor(targets$Sample_Group)), text.col
       cex=0.65, bg="white")

plotMDS(getM(mSetSqFlt), top=1000, gene.selection="common",
        col=pal[factor(targets$Sample_Source)])
legend("right", legend=levels(factor(targets$Sample_Source)), text.cc
       cex=0.7, bg="white")
```

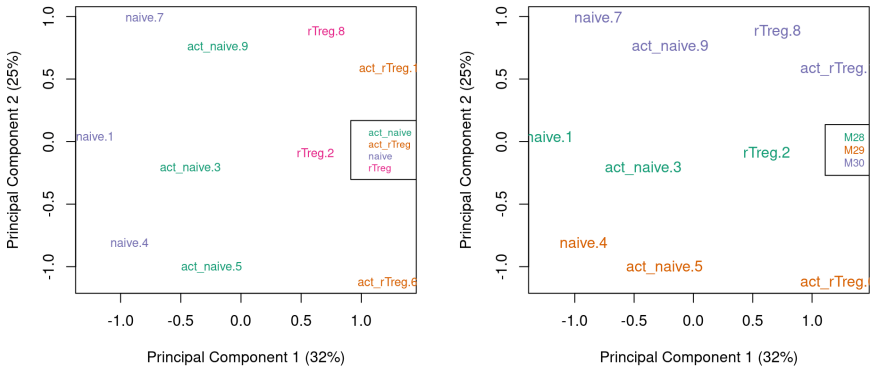


Figure 6: **Removing SNP-affected CpGs probes from the data changes the sample clustering in the MDS plots**

```
par(mfrow=c(1,3))
# Examine higher dimensions to look at other sources of variation
plotMDS(getM(mSetSqFlt), top=1000, gene.selection="common",
        col=pal[factor(targets$Sample_Source)], dim=c(1,3))
legend("right", legend=levels(factor(targets$Sample_Source)), text.cc
       cex=0.7, bg="white")

plotMDS(getM(mSetSqFlt), top=1000, gene.selection="common",
        col=pal[factor(targets$Sample_Source)], dim=c(2,3))
legend("topright", legend=levels(factor(targets$Sample_Source)), text
       cex=0.7, bg="white")

plotMDS(getM(mSetSqFlt), top=1000, gene.selection="common",
        col=pal[factor(targets$Sample_Source)], dim=c(3,4))
legend("right", legend=levels(factor(targets$Sample_Source)), text.cc
       cex=0.7, bg="white")
```

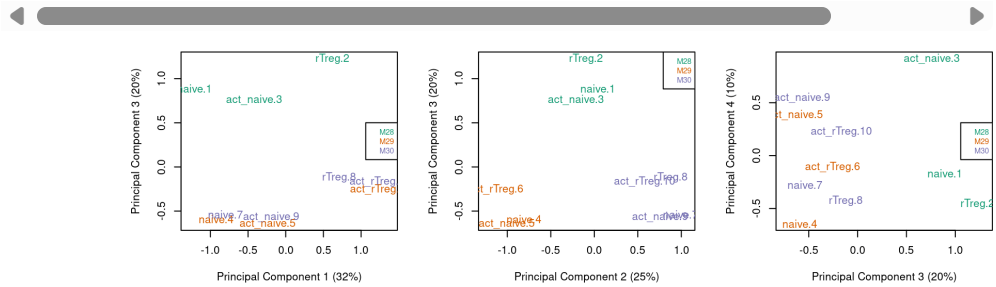


Figure 7: **Examining the higher dimensions of the MDS plots shows that significant inter-individual variation still exists in the second and third principal components**

The next step is to calculate M-values and beta values (Figure 8). As previously mentioned, M-values have nicer statistical properties and are thus better for use in statistical analysis of methylation data whilst beta values are easy to interpret and are thus better for displaying data. A detailed comparison of M-values and beta values was published by Du et al. (2010).

```
# calculate M-values for statistical analysis
mVals <- getM(mSetSqFlt)
head(mVals[,1:5])
```

```
##              naive.1   rTreg.2 act_naive.3   naive.4 act_naive.5
## cg13869341  2.421276  2.515948    2.165745  2.286314    2.109441
## cg24669183  2.169414  2.235964    2.280734  1.632309    2.184435
## cg15560884  1.761176  1.577578    1.597503  1.777486    1.764999
## cg01014490 -3.504268 -3.825119   -5.384735 -4.537864   -4.296526
## cg17505339  3.082191  3.924931    4.163206  3.255373    3.654134
## cg11954957  1.546401  1.912204    1.727910  2.441267    1.618331
```

```
bVals <- getBeta(mSetSqFlt)
head(bVals[,1:5])
```

```
##               naive.1    rTreg.2 act_naive.3    naive.4 act_naive.
## cg13869341 0.84267937 0.85118462   0.8177504 0.82987650  0.8118617
## cg24669183 0.81812908 0.82489238   0.8293297 0.75610281  0.8196732
## cg15560884 0.77219626 0.74903910   0.7516263 0.77417882  0.7726620
## cg01014490 0.08098986 0.06590459   0.0233755 0.04127262  0.0484239
## cg17505339 0.89439216 0.93822870   0.9471357 0.90520570  0.9264130
## cg11954957 0.74495496 0.79008516   0.7681146 0.84450764  0.7543116
```

```
par(mfrow=c(1,2))
densityPlot(bVals, sampGroups=targets$Sample_Group, main="Beta values
            legend=FALSE, xlab="Beta values")
legend("top", legend = levels(factor(targets$Sample_Group)),
       text.col=brewer.pal(8,"Dark2"))
densityPlot(mVals, sampGroups=targets$Sample_Group, main="M-values",
            legend=FALSE, xlab="M values")
legend("topleft", legend = levels(factor(targets$Sample_Group)),
       text.col=brewer.pal(8,"Dark2"))
```

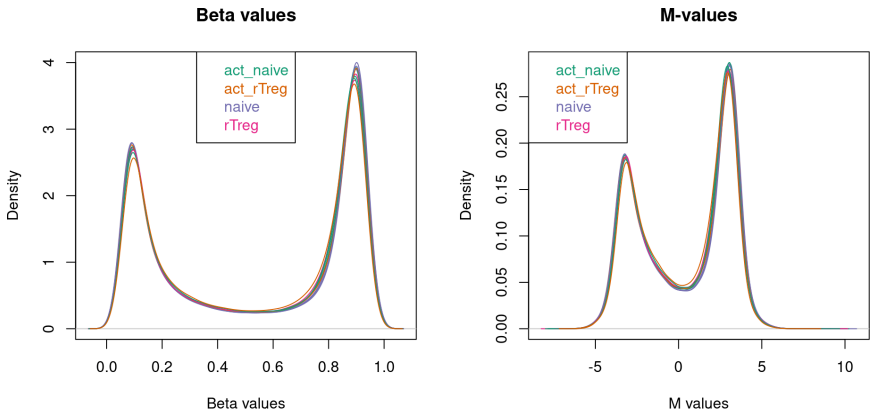


Figure 8: **The distributions of beta and M-values are quite different**
Beta values are constrained between 0 and 1 whilst M-values range between -Inf and Inf.

2.7 Probe-wise differential methylation analysis

The biological question of interest for this particular dataset is to discover differentially methylated probes between the different cell types. However, as was apparent in the MDS plots, there is another factor that we need to take into account when we perform the statistical analysis. In the `targets` file, there is a column called `Sample_Source`, which refers to the individuals that the samples were collected from. In this dataset, each of the individuals contributes more than one cell type. For example, individual M28 contributes `naive`, `rTreg` and `act_naive` samples. Hence, when we specify our design matrix, we need to include two factors: individual and cell type. This style of analysis is called a paired analysis; differences between cell types are calculated *within* each individual, and then these differences are averaged *across* individuals to determine whether there is an overall significant difference in the mean methylation level for each CpG site. The *limma* User's Guide (https://bioconductor.org/packages/release/bioc/vignettes/limma/inst/doc/usersguide.pdf) extensively covers the different types of designs that are commonly used for microarray experiments and how to analyse them in R.

We are interested in pairwise comparisons between the four cell types, taking into account individual to individual variation. We perform this analysis on the matrix of M-values in *limma*, obtaining moderated t-statistics and associated p-values for each CpG site. A convenient way to set up the model when the user has many comparisons of interest that they would like to test is to use a contrasts matrix in conjunction with the design matrix. A contrasts matrix will take linear combinations of the columns of the design matrix corresponding to the comparisons of interest.

Since we are performing hundreds of thousands of hypothesis tests, we need to adjust the p-values for multiple testing. A common procedure for assessing how statistically significant a change in mean levels is between two groups when a very large number of tests is being performed is to assign a cut-off on the false discovery rate (Benjamini and Hochberg 1995), rather than on the unadjusted p-value. Typically 5% FDR is used, and this is interpreted as the researcher willing to accept that from the list of significant differentially methylated CpG sites, 5% will be false discoveries. If the p-values are not adjusted for multiple testing, the number of false discoveries will be unacceptably high. For this dataset, assuming a Type I error rate of 5%, we would expect to see 0.05*439918=21996 statistical significant results for a given comparison, even if there were truly no differentially methylated CpG sites.

Based on a false discovery rate of 5%, there are 3021 significantly differentially methylated CpGs in the `naïve` vs `rTreg` comparison, while `rTreg` vs `act_rTreg` doesn't show any significant differential methylation.

```
# this is the factor of interest
cellType <- factor(targets$Sample_Group)
# this is the individual effect that we need to account for
individual <- factor(targets$Sample_Source)

# use the above to create a design matrix
design <- model.matrix(~0+cellType+individual, data=targets)
colnames(design) <- c(levels(cellType),levels(individual)[-1])

# fit the linear model
fit <- lmFit(mVals, design)
# create a contrast matrix for specific comparisons
contMatrix <- makeContrasts(naive-rTreg,
                            naive-act_naive,
                            rTreg-act_rTreg,
                            act_naive-act_rTreg,
                            levels=design)
contMatrix
```

```
##            Contrasts
## Levels      naive - rTreg naive - act_naive rTreg - act_rTreg
##   act_naive             0               -1                 0
##   act_rTreg             0                0                -1
##   naive                 1                1                 0
##   rTreg                -1                0                 1
##   M29                   0                0                 0
##   M30                   0                0                 0
##            Contrasts
## Levels      act_naive - act_rTreg
##   act_naive                     1
##   act_rTreg                    -1
##   naive                         0
##   rTreg                         0
##   M29                           0
##   M30                           0
```

```
# fit the contrasts
fit2 <- contrasts.fit(fit, contMatrix)
fit2 <- eBayes(fit2)

# look at the numbers of DM CpGs at FDR < 0.05
summary(decideTests(fit2))
```

```
##        naive - rTreg naive - act_naive rTreg - act_rTreg act_naive
## Down            1618              400                0
## NotSig        436895           439291           439918
## Up              1405              227                0
```

We can extract the tables of differentially expressed CpGs for each comparison, ordered by B-statistic by default, using the `topTable` function in *limma*. The B-statistic is the log-odds of differential methylation, first published by Lonnstedt and Speed (Lonnstedt and Speed 2002). To order by p-value, the user can specify `sort.by="p"` ; and in most cases, the ordering based on the p-value and ordering based on the B-statistic will be identical.The results of the analysis for the first comparison, `naive` vs. `rTreg` , can be saved as a `data.frame` by setting `coef=1` . The `coef` parameter explicitly refers to the column in the contrasts matrix which corresponds to the comparison of interest.

```
# get the table of results for the first contrast (naive - rTreg)
ann450kSub <- ann450k[match(rownames(mVals),ann450k$Name),
                      c(1:4,12:19,24:ncol(ann450k))]
DMPs <- topTable(fit2, num=Inf, coef=1, genelist=ann450kSub)
head(DMPs)
```

```
##              chr       pos strand       Name Probe_rs Probe_maf Cp
## cg07499259  chr1  12188502      + cg07499259     <NA>        NA
## cg26992245  chr8  29848579      - cg26992245     <NA>        NA
## cg09747445 chr15  70387268      - cg09747445     <NA>        NA
## cg18808929  chr8  61825469      - cg18808929     <NA>        NA
## cg25015733  chr2  99342986      - cg25015733     <NA>        NA
## cg21179654  chr3 114057297      + cg21179654     <NA>        NA
##            SBE_rs SBE_maf            Islands_Name Relation_to_Isla
## cg07499259   <NA>      NA                                 OpenS
## cg26992245   <NA>      NA                                 OpenS
## cg09747445   <NA>      NA chr15:70387929-70393206           N_Sho
## cg18808929   <NA>      NA  chr8:61822358-61823028           S_She
## cg25015733   <NA>      NA  chr2:99346882-99348177           N_She
## cg21179654   <NA>      NA                                 OpenS
##                                      UCSC_RefGene_Name
## cg07499259                             TNFRSF8;TNFRSF8
## cg26992245
## cg09747445                              TLE3;TLE3;TLE3
## cg18808929
## cg25015733                                       MGAT4A
## cg21179654 ZBTB20;ZBTB20;ZBTB20;ZBTB20;ZBTB20;ZBTB20;ZBTB20
##
## cg07499259
## cg26992245
## cg09747445
## cg18808929
## cg25015733
## cg21179654 NM_001164343;NM_001164346;NM_001164345;NM_001164342;NM_
##                               UCSC_RefGene_Group Phantom DMR E
## cg07499259                            5'UTR;Body
## cg26992245
## cg09747445                         Body;Body;Body
## cg18808929
## cg25015733                                5'UTR
## cg21179654 3'UTR;3'UTR;3'UTR;3'UTR;3'UTR;3'UTR;3'UTR
##                     HMM_Island Regulatory_Feature_Name
## cg07499259 1:12111023-12111225
## cg26992245
## cg09747445
## cg18808929
## cg25015733
## cg21179654                       3:114057192-114057775
##                   Regulatory_Feature_Group DHS     logFC     AveEx
## cg07499259                                    3.654104  2.466521
## cg26992245                                    4.450696 -0.091807
## cg09747445                                   -3.337299 -0.252014
## cg18808929                                   -2.990263  0.775228
## cg25015733                                   -3.054336  0.832801
## cg21179654 Unclassified_Cell_type_specific    2.859016  1.324608
##                 P.Value   adj.P.Val        B
## cg07499259 7.258963e-08 0.005062817 7.454265
## cg26992245 8.603867e-08 0.005062817 7.360267
## cg09747445 8.913981e-08 0.005062817 7.340431
## cg18808929 1.033329e-07 0.005062817 7.256713
```

```
## cg25015733 1.332317e-07 0.005062817 7.109143
## cg21179654 1.361568e-07 0.005062817 7.096322
```

The resulting `data.frame` can easily be written to a CSV file, which can be opened in Excel.

```
write.table(DMPs, file="DMPs.csv", sep=",", row.names=FALSE)
```

It is always useful to plot sample-wise methylation levels for the top differentially methylated CpG sites to quickly ensure the results make sense (Figure 9). If the plots do not look as expected, it is usually an indication of an error in the code, or in setting up the design matrix. It is easier to interpret methylation levels on the beta value scale, so although the analysis is performed on the M-value scale, we visualise data on the beta value scale. The `plotCpg` function in *minfi* is a convenient way to plot the sample-wise beta values stratified by the grouping variable.

```
# plot the top 4 most significantly differentially methylated CpGs
par(mfrow=c(2,2))
sapply(rownames(DMPs)[1:4], function(cpg){
  plotCpg(bVals, cpg=cpg, pheno=targets$Sample_Group, ylab = "Beta va
})
```

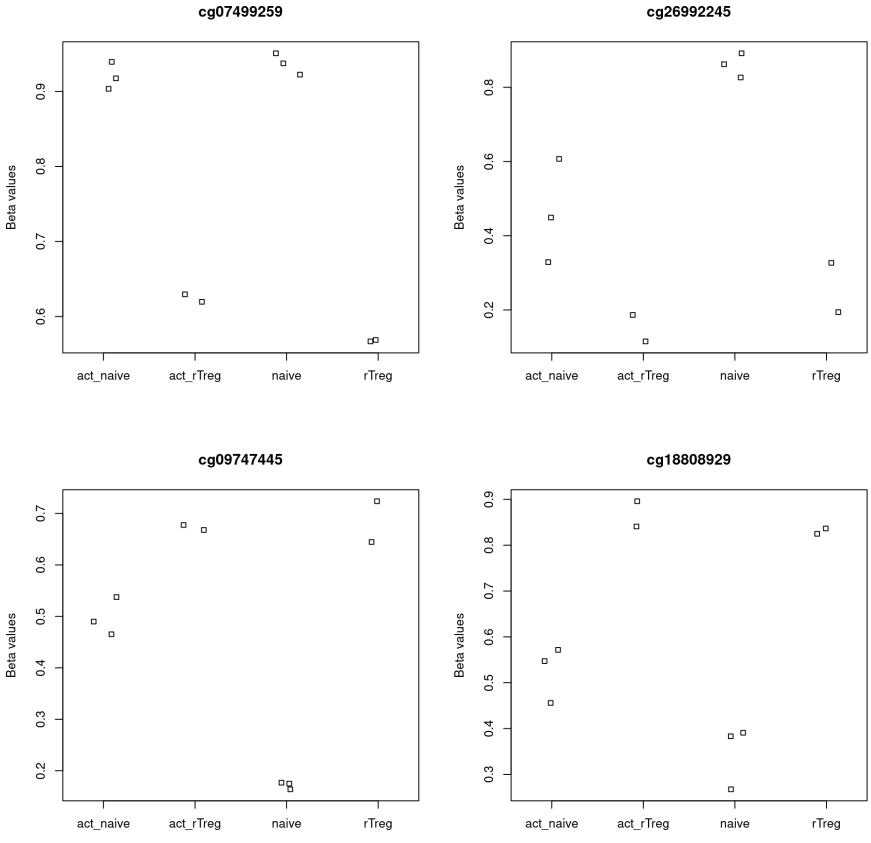


Figure 9: **Plotting the top few differentially methylated CpGs is a good way to check whether the results make sense**

```
## $cg07499259
## NULL
##
## $cg26992245
## NULL
##
## $cg09747445
## NULL
##
## $cg18808929
## NULL
```

2.8 Differential methylation analysis of regions

Although performing a *probe-wise* analysis is useful and informative, sometimes we are interested in knowing whether several proximal CpGs are concordantly differentially methylated, that is, we want to identify differentially methylated *regions*. There are several Bioconductor packages that have functions for identifying differentially methylated regions from 450k data. Some of the most popular are the `dmrFind` function in the charm (http://www.bioconductor.org/packages/release/bioc/html/charm.html) package, which has been somewhat superseded for 450k arrays by the `bumphunter` function in minfi (http://bioconductor.org/packages/release/bioc/html/minfi.html) (Jaffe et al. 2012; Aryee et al. 2014), and, the recently published `dmrcate` in the DMRcate (https://www.bioconductor.org/packages/release/bioc/html/DMRcate.html) package (Peters et al. 2015). They are each based on different statistical methods. In our experience, the `bumphunter` and `dmrFind` functions can be somewhat slow to run unless you have the computer infrastructure to parallelise them, as they use permutations to assign significance. In this workflow, we will perform an analysis using the `dmrcate`. As it is based on *limma*, we can directly use the `design` and `contMatrix` we previously defined.

Firstly, our matrix of M-values is annotated with the relevant information about the probes such as their genomic position, gene annotation, etc. By default, this is done using the `ilmn12.hg19` annotation, but this can be substituted for any argument compatible with the interface provided by the *minfi* package. The *limma* pipeline is then used for differential methylation analysis to calculate moderated t-statistics.

```
myAnnotation <- cpg.annotate(object = mVals, datatype = "array", what
                             analysis.type = "differential", design =
                             contrasts = TRUE, cont.matrix = contMatr
                             coef = "naive - rTreg", arraytype = "450
```

```
## Your contrast returned 3023 individually significant probes. We re
```

```
str(myAnnotation)
```

```
## Formal class 'CpGannotated' [package "DMRcate"] with 1 slot
##   ..@ ranges:Formal class 'GRanges' [package "GenomicRanges"] with
##   .. .. ..@ seqnames       :Formal class 'Rle' [package "S4Vectors
##   .. .. .. .. ..@ values         : Factor w/ 24 levels "chr1","chr
##   .. .. .. .. ..@ lengths        : int [1:24] 42733 31682 23086 18
##   .. .. .. .. ..@ elementMetadata: NULL
##   .. .. .. .. ..@ metadata       : list()
##   .. .. ..@ ranges         :Formal class 'IRanges' [package "IRang
##   .. .. .. .. ..@ start          : int [1:439918] 15865 534242 710
##   .. .. .. .. ..@ width          : int [1:439918] 1 1 1 1 1 1 1 1
##   .. .. .. .. ..@ NAMES          : chr [1:439918] "cg13869341" "cg
##   .. .. .. .. ..@ elementType    : chr "ANY"
##   .. .. .. .. ..@ elementMetadata: NULL
##   .. .. .. .. ..@ metadata       : list()
##   .. .. ..@ strand         :Formal class 'Rle' [package "S4Vectors
##   .. .. .. .. ..@ values         : Factor w/ 3 levels "+","-","*":
##   .. .. .. .. ..@ lengths        : int 439918
##   .. .. .. .. ..@ elementMetadata: NULL
##   .. .. .. .. ..@ metadata       : list()
##   .. .. ..@ seqinfo        :Formal class 'Seqinfo' [package "Genom
##   .. .. .. .. ..@ seqnames   : chr [1:24] "chr1" "chr2" "chr3" "ch
##   .. .. .. .. ..@ seqlengths : int [1:24] NA NA NA NA NA NA NA NA
##   .. .. .. .. ..@ is_circular: logi [1:24] NA NA NA NA NA NA ...
##   .. .. .. .. ..@ genome     : chr [1:24] NA NA NA NA ...
##   .. .. ..@ elementMetadata:Formal class 'DFrame' [package "S4Vect
##   .. .. .. .. ..@ rownames       : NULL
##   .. .. .. .. ..@ nrows          : int 439918
##   .. .. .. .. ..@ elementType    : chr "ANY"
##   .. .. .. .. ..@ elementMetadata: NULL
##   .. .. .. .. ..@ metadata       : list()
##   .. .. .. .. ..@ listData       :List of 4
##   .. .. .. .. .. ..$ stat   : num [1:439918] 0.0489 -2.0773 0.7711
##   .. .. .. .. .. ..$ diff   : num [1:439918] 0.00039 -0.04534 0.01
##   .. .. .. .. .. ..$ ind.fdr: num [1:439918] 0.994 0.565 0.872 0.9
##   .. .. .. .. .. ..$ is.sig : logi [1:439918] FALSE FALSE FALSE FA
##   .. .. ..@ elementType    : chr "ANY"
##   .. .. ..@ metadata       : list()
```

Once we have the relevant statistics for the individual CpGs, we can then use the `dmrcate` function to combine them to identify differentially methylated regions. The main output table `DMRs$results` contains all of the regions found, along with their genomic annotations and p-values.

```
#endif /* NEWSTUFF */
DMRs <- dmrcate(myAnnotation, lambda=1000, C=2)
results.ranges <- extractRanges(DMRs)
results.ranges
```

```
## GRanges object with 545 ranges and 8 metadata columns:
##         seqnames              ranges strand |   no.cpgs min_smooth
##            <Rle>           <IRanges>  <Rle> | <integer>        <nu
##     [1]    chr17   57915665-57918682      * |        12      4.943
##     [2]     chr3 114012316-114012912      * |         5     1.6301
##     [3]    chr18   21452730-21453131      * |         7     5.7724
##     [4]    chr17   74639731-74640078      * |         6      9.628
##     [5]     chrX   49121205-49122718      * |         6      6.757
##     ...      ...                 ...    ... .       ...
##   [541]     chr2   43454761-43455103      * |        14      1.290
##   [542]     chr6   31832650-31833452      * |        18      2.467
##   [543]     chrX   43741310-43742501      * |         9      5.270
##   [544]     chr6 144385771-144387124      * |        22      2.852
##   [545]     chr2   25141532-25142229      * |         8      4.314
##            Stouffer      HMFDR      Fisher   maxdiff   meandiff
##           <numeric>  <numeric>   <numeric> <numeric>  <numeric>
##     [1] 6.60666e-10 0.02351388 6.57544e-08  0.398286   0.313161
##     [2] 1.51038e-07 0.00707524 1.39235e-06  0.543428   0.425162
##     [3] 7.65545e-07 0.01239758 1.85082e-06 -0.386747  -0.254609
##     [4] 1.52368e-07 0.01403323 2.60145e-06 -0.252864  -0.195190
##     [5] 2.92694e-07 0.01163337 3.55872e-06  0.452909   0.300624
##     ...         ...        ...         ...       ...        ...
##   [541]    0.967707  0.1532340    0.620701 -0.218836 -0.0427390
##   [542]    0.886282  0.2310998    0.647328  0.153367  0.0490080
##   [543]    0.914407  0.0431746    0.655714  0.413832  0.0558174
##   [544]    0.996631  0.0796880    0.690460  0.325422  0.0449451
##   [545]    0.992418  0.0567769    0.748698  0.282058  0.0314244
##         overlapping.genes
##               <character>
##     [1]       VMP1, MIR21
##     [2]             TIGIT
##     [3]             LAMA3
##     [4]        ST6GALNAC1
##     [5]             FOXP3
##     ...               ...
##   [541]             THADA
##   [542]           SLC44A4
##   [543]              MAOB
##   [544]              <NA>
##   [545]             ADCY3
##   -------
##   seqinfo: 23 sequences from an unspecified genome; no seqlengths
```

As for the probe-wise analysis, it is advisable to visualise the results to ensure that they make sense. The regions can easily be viewed using the `DMR.plot` function provided in the *DMRcate* package (Figure 10).

```
# set up the grouping variables and colours
groups <- pal[1:length(unique(targets$Sample_Group))]
names(groups) <- levels(factor(targets$Sample_Group))
cols <- groups[as.character(factor(targets$Sample_Group))]
```

```
# draw the plot for the top DMR
par(mfrow=c(1,1))
DMR.plot(ranges = results.ranges, dmr = 2, CpGs = bVals, phen.col = c
         what = "Beta", arraytype = "450K", genome = "hg19")
```

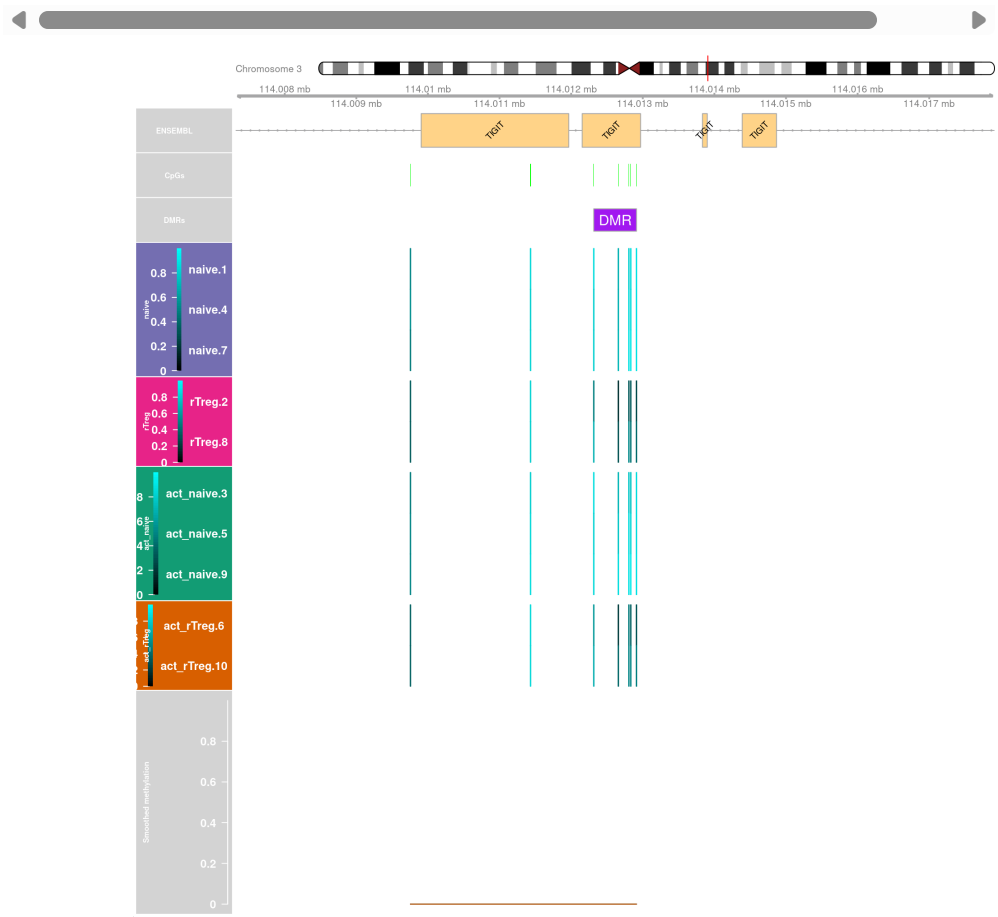


Figure 10: **DMRcate allows you to quickly visualise DMRs in their genomic context**
By default, the plot shows the location of the DMR in the genome, the position of any genes that are nearby, the base pair positions of the CpG probes, the methylation levels of the individual samples as a heatmap and the mean methylation levels for the various sample groups in the experiment. This plot shows one of the DMRs identified by the DMRcate analysis.

2.9 Customising visualisations of methylation data

The *Gviz* package offers powerful functionality for plotting methylation data in its genomic context. The package vignette (https://bioconductor.org/packages/release/bioc/vignettes/Gviz/inst/doc/Gviz.pdf) is very extensive and covers the various types of plots that can be produced using the *Gviz* framework. We will plot one of the differentially methylated regions from the *DMRcate* analysis to demonstrate the type of visualisations that can be created (Figure 11).

We will first set up the genomic region we would like to plot by extracting the genomic coordinates of one of the differentially methylated regions.

```
# indicate which genome is being used
gen <- "hg19"
# the index of the DMR that we will plot
dmrIndex <- 1
# extract chromosome number and location from DMR results
chrom <- as.character(seqnames(results.ranges[dmrIndex]))
start <- as.numeric(start(results.ranges[dmrIndex]))
end <- as.numeric(end(results.ranges[dmrIndex]))
# add 25% extra space to plot
minbase <- start - (0.25*(end-start))
maxbase <- end + (0.25*(end-start))
```

Next, we will add some genomic annotations of interest such as the locations of CpG islands and DNAseI hypersensitive sites; this can be any feature or genomic annotation of interest that you have data available for. The CpG islands data was generated using the method published by Wu et al. (2010); the DNaseI hypersensitive site data was obtained from the UCSC Genome Browser (https://genome.ucsc.edu/cgi-bin/hgTables).

```
# CpG islands
islandHMM <- read.csv(paste0(dataDirectory,
                             "/model-based-cpg-islands-hg19-chr17.txt
                      sep="\t", stringsAsFactors=FALSE, header=FALSE)
head(islandHMM)
```

```
##                V1     V2     V3   V4  V5   V6    V7    V8
## 1 chr17_ctg5_hap1   8935  10075 1141 129  815 0.714 0.887
## 2 chr17_ctg5_hap1  64252  64478  227  30  165 0.727 1.014
## 3 chr17_ctg5_hap1  87730  89480 1751 135 1194 0.682 0.663
## 4 chr17_ctg5_hap1  98265  98591  327  29  226 0.691 0.744
## 5 chr17_ctg5_hap1 120763 125451 4689 359 3032 0.647 0.733
## 6 chr17_ctg5_hap1 146257 146607  351  19  231 0.658 0.500
```

```
islandData <- GRanges(seqnames=Rle(islandHMM[,1]),
                      ranges=IRanges(start=islandHMM[,2], end=islandH
                      strand=Rle(strand(rep("*",nrow(islandHMM)))))
islandData
```

```
## GRanges object with 3456 ranges and 0 metadata columns:
##                 seqnames            ranges strand
##                    <Rle>         <IRanges>  <Rle>
##      [1] chr17_ctg5_hap1        8935-10075      *
##      [2] chr17_ctg5_hap1       64252-64478      *
##      [3] chr17_ctg5_hap1       87730-89480      *
##      [4] chr17_ctg5_hap1       98265-98591      *
##      [5] chr17_ctg5_hap1     120763-125451      *
##      ...             ...               ...    ...
##   [3452]           chr17 81147380-81147511      *
##   [3453]           chr17 81147844-81148321      *
##   [3454]           chr17 81152612-81153665      *
##   [3455]           chr17 81156194-81156512      *
##   [3456]           chr17 81162945-81165532      *
##   -------
##   seqinfo: 5 sequences from an unspecified genome; no seqlengths


# DNAseI hypersensitive sites
dnase <- read.csv(paste0(dataDirectory,"/wgEncodeRegDnaseClusteredV3c
                 sep="\t",stringsAsFactors=FALSE,header=FALSE)
head(dnase)
```

```
##      V1   V2   V3 V4  V5 V6
## 1 chr17  125  335  7 444  7                    84,83,88,90,77,87,8
## 2 chr17  685  835  1 150  1                                      8
## 3 chr17 2440 2675 13 410 13 0,30,102,104,38,47,61,68,122,1,51,73,7
## 4 chr17 3020 3170  1 247  1                                     12
## 5 chr17 3740 3890  2 161  2                                  71,7
## 6 chr17 5520 6110  4 241  5                         17,19,25,16,1
##                                            V8
## 1                   328,208,444,218,109,171,191,
## 2                                        150,
## 3 204,410,301,206,46,48,84,164,85,12,98,215,146,
## 4                                        247,
## 5                                    108,161,
## 6                             241,185,239,26,52,
```

```
dnaseData <- GRanges(seqnames=dnase[,1],
                     ranges=IRanges(start=dnase[,2], end=dnase[,3]),
                     strand=Rle(rep("*",nrow(dnase))),
                     data=dnase[,5])
dnaseData
```

```
## GRanges object with 74282 ranges and 1 metadata column:
##           seqnames            ranges strand |      data
##              <Rle>         <IRanges>  <Rle> | <integer>
##       [1]    chr17           125-335      * |       444
##       [2]    chr17           685-835      * |       150
##       [3]    chr17         2440-2675      * |       410
##       [4]    chr17         3020-3170      * |       247
##       [5]    chr17         3740-3890      * |       161
##       ...      ...               ...    ... .       ...
##   [74278]    chr17 81153140-81153350      * |       574
##   [74279]    chr17 81153580-81153810      * |       208
##   [74280]    chr17 81185540-81185750      * |       326
##   [74281]    chr17 81188880-81189090      * |       209
##   [74282]    chr17 81194900-81195115      * |       185
##   -------
##   seqinfo: 1 sequence from an unspecified genome; no seqlengths
```

Now, set up the ideogram, genome and RefSeq tracks that will provide context for our methylation data.

```
iTrack <- IdeogramTrack(genome = gen, chromosome = chrom, name="")
gTrack <- GenomeAxisTrack(col="black", cex=1, name="", fontcolor="bla
rTrack <- UcscTrack(genome=gen, chromosome=chrom, track="NCBI RefSeq"
                    from=minbase, to=maxbase, trackType="GeneRegionTr
                    rstarts="exonStarts", rends="exonEnds", gene="nam
                    symbol="name2", transcript="name", strand="stranc
                    fill="darkblue",stacking="squish", name="RefSeq",
                    showId=TRUE, geneSymbol=TRUE)
```

```
## Warning in .local(x, ...): 'track' parameter is deprecated now you
##                 Use ucscTables(genome, track) to retrieve the list
## Warning in .local(x, ...): 'track' parameter is deprecated now you
##                 Use ucscTables(genome, track) to retrieve the list
```

Ensure that the methylation data is ordered by chromosome and base position.

```
ann450kOrd <- ann450kSub[order(ann450kSub$chr,ann450kSub$pos),]
head(ann450kOrd)
```

```
## DataFrame with 6 rows and 22 columns
##                    chr       pos      strand        Name    Probe_
##            <character> <integer> <character> <character> <characte
## cg13869341        chr1     15865           +  cg13869341
## cg24669183        chr1    534242           -  cg24669183   rs66807
## cg15560884        chr1    710097           +  cg15560884
## cg01014490        chr1    714177           -  cg01014490
## cg17505339        chr1    720865           -  cg17505339
## cg11954957        chr1    758829           +  cg11954957 rs1154984
##                 CpG_rs   CpG_maf      SBE_rs   SBE_maf       Islar
##            <character> <numeric> <character> <numeric>        <cha
## cg13869341          NA        NA          NA        NA
## cg24669183          NA        NA          NA        NA chr1:533219
## cg15560884          NA        NA          NA        NA chr1:713984
## cg01014490          NA        NA          NA        NA chr1:713984
## cg17505339          NA        NA          NA        NA
## cg11954957          NA        NA          NA        NA chr1:762416
##            Relation_to_Island UCSC_RefGene_Name UCSC_RefGene_Acces
##                   <character>       <character>            <charac
## cg13869341            OpenSea             WASH5P              NR_02
## cg24669183            S_Shore
## cg15560884            N_Shelf
## cg01014490             Island
## cg17505339            OpenSea
## cg11954957            N_Shelf
##            UCSC_RefGene_Group     Phantom         DMR    Enhancer
##                   <character> <character> <character> <character>
## cg13869341               Body
## cg24669183
## cg15560884
## cg01014490
## cg17505339
## cg11954957
##                 HMM_Island Regulatory_Feature_Name Regulatory_Feat
##                <character>             <character>              <c
## cg13869341
## cg24669183 1:523025-524193
## cg15560884
## cg01014490 1:703784-704410         1:713802-715219      Promoter_A
## cg17505339
## cg11954957
##                        DHS
##            <character>
## cg13869341
## cg24669183
## cg15560884
## cg01014490
## cg17505339
## cg11954957
```

```
bValsOrd <- bVals[match(ann450kOrd$Name,rownames(bVals)),]
head(bValsOrd)
```

```
##               naive.1    rTreg.2 act_naive.3    naive.4 act_naive.
## cg13869341 0.84267937 0.85118462   0.8177504 0.82987650  0.8118617
## cg24669183 0.81812908 0.82489238   0.8293297 0.75610281  0.8196732
## cg15560884 0.77219626 0.74903910   0.7516263 0.77417882  0.7726620
## cg01014490 0.08098986 0.06590459   0.0233755 0.04127262  0.0484239
## cg17505339 0.89439216 0.93822870   0.9471357 0.90520570  0.9264130
## cg11954957 0.74495496 0.79008516   0.7681146 0.84450764  0.7543116
##              naive.7    rTreg.8 act_naive.9 act_rTreg.10
## cg13869341 0.8891851 0.88537940  0.90916748   0.88334231
## cg24669183 0.7903763 0.85304116  0.80930568   0.80979554
## cg15560884 0.7658623 0.75909061  0.78099397   0.78569274
## cg01014490 0.0245281 0.02832358  0.07740468   0.04640659
## cg17505339 0.8889361 0.87205348  0.90099782   0.93508348
## cg11954957 0.7832207 0.84929777  0.84719430   0.83350220
```

Create the data tracks using the appropriate track type for each data type.

```
# create genomic ranges object from methylation data
cpgData <- GRanges(seqnames=Rle(ann450kOrd$chr),
                   ranges=IRanges(start=ann450kOrd$pos, end=ann450kOr
                   strand=Rle(rep("*",nrow(ann450kOrd))),
                   betas=bValsOrd)
# extract data on CpGs in DMR
cpgData <- subsetByOverlaps(cpgData, results.ranges[dmrIndex])

# methylation data track
methTrack <- DataTrack(range=cpgData, groups=targets$Sample_Group,ger
                       chromosome=chrom, ylim=c(-0.05,1.05), col=pal,
                       type=c("a","p"), name="DNA Meth.\n(beta value)
                       background.panel="white", legend=TRUE, cex.tit
                       cex.axis=0.8, cex.legend=0.8)
# CpG island track
islandTrack <- AnnotationTrack(range=islandData, genome=gen, name="Cp
                               chromosome=chrom,fill="darkgreen")

# DNaseI hypersensitive site data track
dnaseTrack <- DataTrack(range=dnaseData, genome=gen, name="DNAseI",
                        type="gradient", chromosome=chrom)

# DMR position data track
dmrTrack <- AnnotationTrack(start=start, end=end, genome=gen, name="D
                            chromosome=chrom,fill="darkred")
```

Set up the track list and indicate the relative sizes of the different tracks. Finally, draw the plot using the `plotTracks` function (Figure 11).

```
tracks <- list(iTrack, gTrack, methTrack, dmrTrack, islandTrack, dnas
               rTrack)
sizes <- c(2,2,5,2,2,2,3) # set up the relative sizes of the tracks
plotTracks(tracks, from=minbase, to=maxbase, showTitle=TRUE, add53=TR
           add35=TRUE, grid=TRUE, lty.grid=3, sizes = sizes, length(t
```

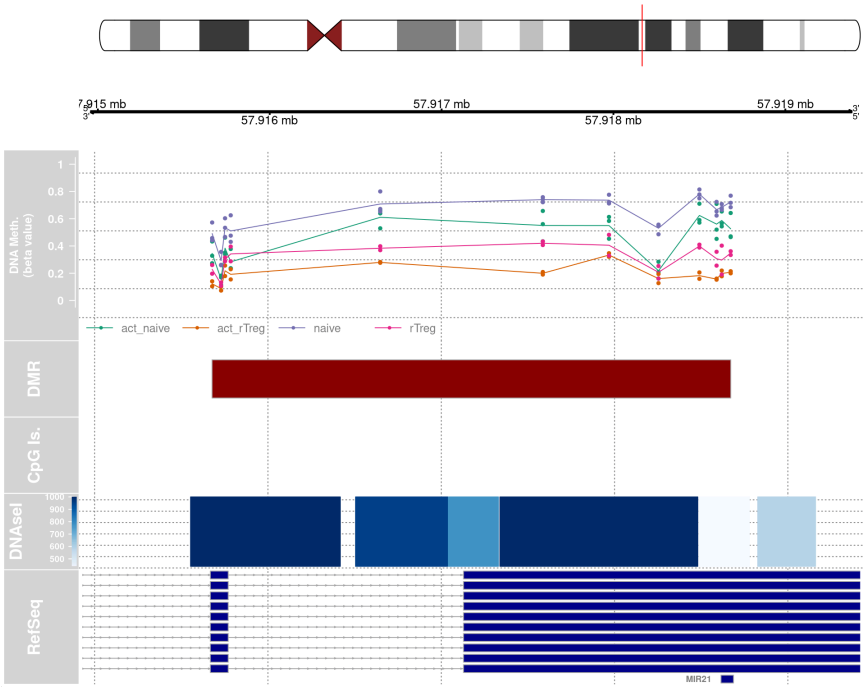


Figure 11: **The Gviz package provides extensive functionality for customising plots of genomic regions**
This plot shows one of the DMRs identified by the DMRcate analysis.

3 Additional analyses

3.1 Gene ontology testing

Once you have performed a differential methylation analysis, there may be a very long list of significant CpG sites to interpret. One question a researcher may have is, “which gene pathways are over-represented for differentially methylated CpGs?” In some cases it is relatively straightforward to link the top differentially methylated CpGs to genes that make biological sense in terms of the cell types or samples being studied, but there may be many thousands of CpGs significantly differentially methylated. In order to gain an understanding of the biological processes that the differentially methylated CpGs may be involved in, we can perform gene ontology or KEGG pathway analysis using the `gometh` function in the *missMethyl* package (Phipson, Maksimovic, and Oshlack 2016).

Let us consider the first comparison, naive vs rTreg, with the results of the analysis in the `DMPs` table. The `gometh` function takes as input a character vector of the names (e.g. cg20832020) of the significant CpG sites, and optionally, a character vector of all CpGs tested. This is recommended particularly if extensive filtering of the CpGs has been performed prior to analysis. For gene ontology testing (default), the user can specify `collection="GO"`. For testing KEGG pathways, specify `collection="KEGG"`. In the `DMPs` table, the `Name` column corresponds to the CpG name. We will select all CpG sites that have adjusted p-value of less than 0.05.

```
# Get the significant CpG sites at less than 5% FDR
sigCpGs <- DMPs$Name[DMPs$adj.P.Val<0.05]
# First 10 significant CpGs
sigCpGs[1:10]
```

```
##  [1] "cg07499259" "cg26992245" "cg09747445" "cg18808929" "cg250157
##  [6] "cg21179654" "cg26280976" "cg16943019" "cg10898310" "cg251303
```

```
# Total number of significant CpGs at 5% FDR
length(sigCpGs)
```

```
## [1] 3023
```

```
# Get all the CpG sites used in the analysis to form the background
all <- DMPs$Name
# Total number of CpG sites tested
length(all)
```

```
## [1] 439918
```

The `gometh` function takes into account the varying numbers of CpGs associated with each gene on the Illumina methylation arrays. For the 450k array, the numbers of CpGs mapping to genes can vary from as few as 1 to as many as 1200. The genes that have more CpGs associated with them will have a higher probability of being identified as differentially methylated compared to genes with fewer CpGs. We can look at this bias in the data by specifying `plot=TRUE` in the call to `gometh` (Figure 12).

```
par(mfrow=c(1,1))
gst <- gometh(sig.cpg=sigCpGs, all.cpg=all, plot.bias=TRUE)
```

```
## All input CpGs are used for testing.
```

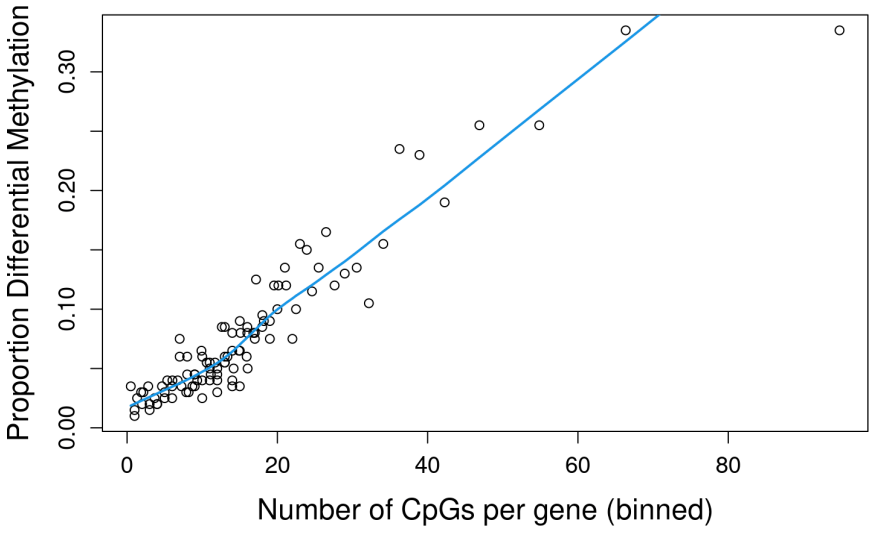


Figure 12: **Bias resulting from different numbers of CpG probes in different genes**

The `gst` object is a `data.frame` with each row corresponding to the GO category being tested. Note that the warning regarding multiple symbols will always be displayed as there are genes that have more than one alias, however it is not a cause for concern.

The top 20 gene ontology categories can be displayed using the `topGSA` function. For KEGG pathway analysis, the `topGSA` function will also display the top 20 enriched pathways.

```
# Top 10 GO categories
topGSA(gst, number=10)
```

```
##            ONTOLOGY                                          TERM
## GO:0002376       BP                       immune system process 2
## GO:0002682       BP          regulation of immune system process 1
## GO:0046649       BP                       lymphocyte activation
## GO:0006955       BP                             immune response 1
## GO:0042110       BP                          T cell activation
## GO:0001775       BP                            cell activation 1
## GO:0002684       BP positive regulation of immune system process 1
## GO:0050776       BP               regulation of immune response
## GO:0045321       BP                        leukocyte activation
## GO:0030097       BP                                 hemopoiesis
##                     P.DE          FDR
## GO:0002376 3.334564e-29 7.508105e-25
## GO:0002682 2.549262e-24 2.869959e-20
## GO:0046649 7.502123e-21 5.630594e-17
## GO:0006955 1.575559e-20 8.868820e-17
## GO:0042110 4.880481e-19 2.197778e-15
## GO:0001775 9.018035e-18 3.384168e-14
## GO:0002684 1.207985e-17 3.885571e-14
## GO:0050776 4.264050e-17 1.160217e-13
## GO:0045321 4.637569e-17 1.160217e-13
## GO:0030097 1.940124e-16 4.368383e-13
```

From the output we can see many of the top GO categories correspond to immune system and T cell processes, which is unsurprising as the cell types being studied form part of the immune system. Typically, we consider GO categories that have associated false discovery rates of less than 5% to be statistically significant. If there aren't any categories that achieve this significance it may be useful to scan the top 5 or 10 highly ranked GO categories to gain some insight into the biological system.

The `gometh` function only tests GO and KEGG pathways. For a more generalised version of gene set testing for methylation data where the user can specify the gene set to be tested, the `gsameth` function can be used. To display the top 20 pathways, `topGSA` can be called. `gsameth` accepts a single gene set, or a list of gene sets. The gene identifiers in the gene set must be Entrez Gene IDs. To demonstrate `gsameth`, we are using the curated genesets (C2) from the Broad Institute Molecular signatures database (http://software.broadinstitute.org/gsea/msigdb). These can be downloaded as an `RData` object from the WEHI Bioinformatics website (http://bioinf.wehi.edu.au/software/MSigDB/).

```
# load Broad human curated (C2) gene sets
load(paste(dataDirectory,"human_c2_v5.rdata",sep="/"))
# perform the gene set test(s)
gsa <- gsameth(sig.cpg=sigCpGs, all.cpg=all, collection=Hs.c2)


## All input CpGs are used for testing.


# top 10 gene sets
topGSA(gsa, number=10)


##                                             N  DE         P.DE
## ZHENG_BOUND_BY_FOXP3                      487 136 5.327905e-28 2.5
## JAATINEN_HEMATOPOIETIC_STEM_CELL_DN       222  57 1.664931e-15 3.9
## MARTENS_BOUND_BY_PML_RARA_FUSION          450 104 1.133048e-13 1.7
## SMID_BREAST_CANCER_NORMAL_LIKE_UP         455  90 3.646310e-13 4.3
## PILON_KLF1_TARGETS_DN                    1956 258 3.855510e-12 3.6
## LEE_EARLY_T_LYMPHOCYTE_DN                  55  25 1.458268e-11 1.1
## MARSON_BOUND_BY_FOXP3_UNSTIMULATED       1216 165 9.900837e-11 6.6
## ZHENG_FOXP3_TARGETS_IN_THYMUS_UP          193  51 2.113630e-10 1.2
## DEURIG_T_CELL_PROLYMPHOCYTIC_LEUKEMIA_DN  310  62 6.947310e-10 3.6
## REACTOME_IMMUNE_SYSTEM                    861 124 9.252396e-10 4.3
```

While gene set testing is useful for providing some biological insight in terms of what pathways might be affected by abberant methylation, care should be taken not to over-interpret the results. Gene set testing should be used for the purpose of providing some biological insight that ideally would be tested and validated in further laboratory experiments. It is important to keep in mind that we are not observing gene level activity such as in RNA-Seq experiments, and that we have had to take an extra step to associate CpGs with genes.

3.2 Differential variability

Rather than testing for differences in mean methylation, we may be interested in testing for differences between group variances. For example, it has been hypothesised that highly variable CpGs in cancer may contribute to tumour heterogeneity (Hansen et al. 2011). Hence we may be interested in CpG sites that are consistently methylated in one group, but variably methylated in another group.

Sample size is an important consideration when testing for differentially variable CpG sites. In order to get an accurate estimate of the group variances, larger sample sizes are required than for estimating group means. A good rule of thumb is to have at least ten samples in each group (Phipson and Oshlack 2014). To demonstrate testing for differentially variable CpG sites, we will use a publicly available dataset on ageing GSE30870 (http://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE30870), where whole blood samples were collected from 18 centenarians and 18 newborns and profiled for methylation on the 450k array (Heyn et al. 2012). The data (`age.rgSet`) and sample information (`age.targets`) have been included as an R data object in both the workflow package or the data archive you downloaded from figshare

(https://figshare.com/s/7a37f43c0ca2fec4669e). We can load the data using the `load` command, after which it needs to be normalised and filtered as previously described.

```
load(file.path(dataDirectory,"ageData.RData"))

# calculate detection p-values
age.detP <- detectionP(age.rgSet)

# pre-process the data after excluding poor quality samples
age.mSetSq <- preprocessQuantile(age.rgSet)


## [preprocessQuantile] Mapping to genome.


## [preprocessQuantile] Fixing outliers.


## [preprocessQuantile] Quantile normalizing.


# add sex information to targets information
age.targets$Sex <- getSex(age.mSetSq)$predictedSex

# ensure probes are in the same order in the mSetSq and detP objects
age.detP <- age.detP[match(featureNames(age.mSetSq),rownames(age.detP
# remove poor quality probes
keep <- rowSums(age.detP < 0.01) == ncol(age.detP)
age.mSetSqFlt <- age.mSetSq[keep,]

# remove probes with SNPs at CpG or single base extension (SBE) site
age.mSetSqFlt <- dropLociWithSnps(age.mSetSqFlt, snps = c("CpG", "SBE

# remove cross-reactive probes
keep <- !(featureNames(age.mSetSqFlt) %in% xReactiveProbes$TargetID)
age.mSetSqFlt <- age.mSetSqFlt[keep,]
```

As this dataset contains samples from both males and females, we can use it to demonstrate the effect of removing sex chromosome probes on the data. The MDS plots below show the relationship between the samples in the ageing dataset before and after sex chromosome probe removal (Figure 13). It is apparent that before the removal of sex chromosome probes, the sample cluster based on sex in the second principal component. When the sex chromosome probes are removed, age is the largest source of variation present and the male and female samples no longer form separate clusters.

```
# tag sex chromosome probes for removal
keep <- !(featureNames(age.mSetSqFlt) %in% ann450k$Name[ann450k$chr %
                                                          c("chrX",

age.pal <- brewer.pal(8,"Set1")
par(mfrow=c(1,2))
plotMDS(getM(age.mSetSqFlt), top=1000, gene.selection="common",
        col=age.pal[factor(age.targets$Sample_Group)], labels=age.tar
        main="With Sex CHR Probes")
legend("topleft", legend=levels(factor(age.targets$Sample_Group)),
       text.col=age.pal)

plotMDS(getM(age.mSetSqFlt[keep,]), top=1000, gene.selection="common"
        col=age.pal[factor(age.targets$Sample_Group)], labels=age.tar
        main="Without Sex CHR Probes")
legend("top", legend=levels(factor(age.targets$Sample_Group)),
       text.col=age.pal)
```

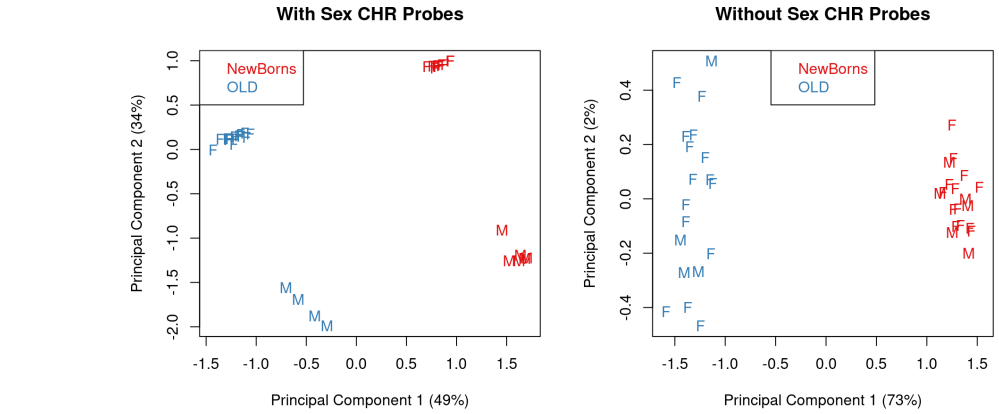


Figure 13: **When samples from both males and females are included in a study, sex is usually the largest source of variation in methylation data**

```
# remove sex chromosome probes from data
age.mSetSqFlt <- age.mSetSqFlt[keep,]
```

We can test for differentially variable CpGs using the `varFit` function in the *missMethyl* package. The syntax for specifying which groups we are interested in testing is slightly different to the standard way a model is specified in `limma`, particularly for designs where an intercept is fitted (see *missMethyl* vignette (https://www.bioconductor.org/packages/release/bioc/vignettes/missMethyl/inst/doc/missMethyl.pdf) for further details). For the ageing data, the design matrix includes an intercept term, and a term for age. The `coef` argument in the `varFit` function indicates which columns of the design matrix correspond to the intercept and grouping factor. Thus, for the ageing dataset we set `coef=c(1,2)`. Note that design matrices without intercept terms are permitted, with specific contrasts tested using the `contrasts.varFit` function.

```
# get M-values for analysis
age.mVals <- getM(age.mSetSqFlt)

design <- model.matrix(~factor(age.targets$Sample_Group))
# Fit the model for differential variability
# specifying the intercept and age as the grouping factor
fitvar <- varFit(age.mVals, design = design, coef = c(1,2))

# Summary of differential variability
summary(decideTests(fitvar))
```

```
##        (Intercept) factor(age.targets$Sample_Group)OLD
## Down             0                                1325
## NotSig       11441                              393451
## Up          417787                               34452
```

```
topDV <- topVar(fitvar, coef=2)
# Top 10 differentially variable CpGs between old vs. newborns
topDV
```

```
##            SampleVar LogVarRatio DiffLevene         t      P.Value
## cg19078576 1.1128910    3.746586  0.8539180  7.006476 6.234780e-10
## cg11661000 0.5926226    3.881306  0.8413614  6.945711 8.176807e-10
## cg07065220 1.0111380    4.181802  0.9204407  6.840327 1.306987e-09
## cg05995465 1.4478673   -5.524284 -1.3035981 -6.708321 2.346207e-09
## cg18091046 1.1121511    3.564282  1.0983340  6.679920 2.659957e-09
## cg05491001 0.9276904    3.869760  0.7118591  6.675892 2.707701e-09
## cg05542681 1.0287320    3.783637  0.9352814  6.635588 3.234735e-09
## cg02726803 0.3175570    4.063650  0.6418968  6.607508 3.660822e-09
## cg08362283 1.0028907    4.783899  0.6970960  6.564472 4.424094e-09
## cg18160402 0.5624192    3.716228  0.5907985  6.520508 5.366535e-09
```

Similarly to the differential methylation analysis, is it useful to plot sample-wise beta values for the differentially variable CpGs to ensure the significant results are not driven by artifacts or outliers (Figure 14).

```
# get beta values for ageing data
age.bVals <- getBeta(age.mSetSqFlt)

par(mfrow=c(2,2))
sapply(rownames(topDV)[1:4], function(cpg){
  plotCpg(age.bVals, cpg=cpg, pheno=age.targets$Sample_Group,
          ylab = "Beta values")
})
```

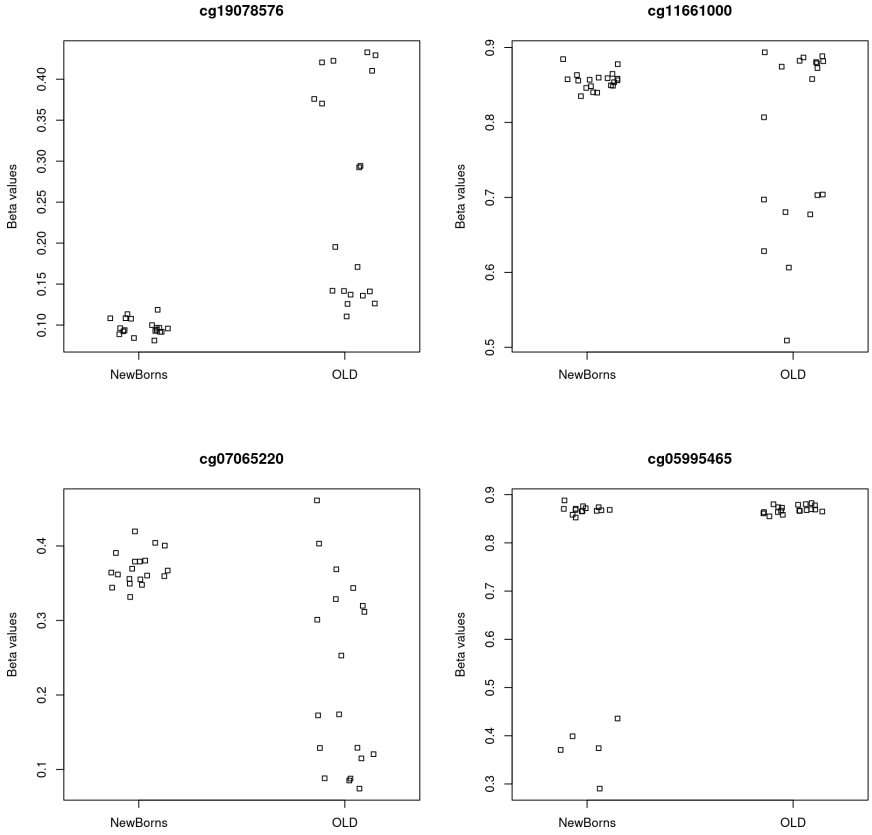


Figure 14: **As for DMPs, it is useful to plot the top few differentially variable CpGs to check that the results make sense**

An example of testing for differential variability when the design matrix does not have an intercept term is detailed in the *missMethyl* vignette (%22http://www.bioconductor.org/packages/release/bioc/vignettes/missMethyl/inst/doc/missMethyl.pdf%22).

3.3 Cell type composition

As methylation is cell type specific and methylation arrays provide CpG methylation values for a population of cells, biological findings from samples that are comprised of a mixture of cell types, such as blood, can be confounded with cell type composition (Jaffe and Irizarry 2014). The *minfi* function `estimateCellCounts` facilitates the estimation of the level of confounding between phenotype and cell type composition in a set of samples. The function uses a modified version of the method published by Houseman et al. (2012) and the package `FlowSorted.Blood.450k`, which contains 450k methylation data from sorted blood cells, to estimate the cell type composition of blood samples.

```
# load sorted blood cell data package
library(FlowSorted.Blood.450k)
# ensure that the "Slide" column of the rgSet pheno data is numeric
# to avoid "estimateCellCounts" error
pData(age.rgSet)$Slide <- as.numeric(pData(age.rgSet)$Slide)
# estimate cell counts
cellCounts <- estimateCellCounts(age.rgSet)
```

```
# plot cell type proportions by age
par(mfrow=c(1,1))
a = cellCounts[age.targets$Sample_Group == "NewBorns",]
b = cellCounts[age.targets$Sample_Group == "OLD",]
boxplot(a, at=0:5*3 + 1, xlim=c(0, 18), ylim=range(a, b), xaxt="n",
        col=age.pal[1], main="", ylab="Cell type proportion")
boxplot(b, at=0:5*3 + 2, xaxt="n", add=TRUE, col=age.pal[2])
axis(1, at=0:5*3 + 1.5, labels=colnames(a), tick=TRUE)
legend("topleft", legend=c("NewBorns","OLD"), fill=age.pal)
```

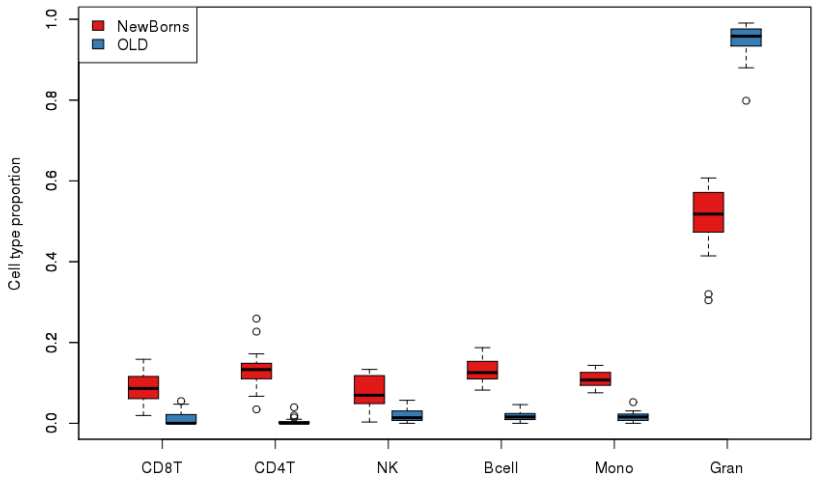


Figure 15: **Cell type composition estimation**
If samples come from a population of mixed cells such as blood, it is advisable to check for potential confounding between differences in cell type proportions and the factor of interest.

As reported by Jaffe and Irizarry (2014), the preceding plot demonstrates that differences in blood cell type proportions are strongly confounded with age in this dataset (Figure 15). Performing cell composition estimation can alert you to potential issues with confounding when analysing a mixed cell type dataset. Based on the results, some type of adjustment for cell type composition may be considered, although a naive cell type adjustment is not recommended. Jaffe and Irizarry (2014) outline several strategies for dealing with cell type composition issues.

4 Discussion

Here we present a commonly used workflow for methylation array analysis based on a series of Bioconductor packages. While we have not included all the possible functions or analysis options that are available for detecting differential methylation, we have demonstrated a common and well used workflow that we regularly use in our own analysis. Specifically, we have not demonstrated more complex types of analyses such as removing unwanted variation in a differential methylation study (Maksimovic et al. 2015; Leek et al. 2012; Teschendorff, Zhuang, and Widschwendter 2011), block finding (Hansen et al. 2011; Aryee et al. 2014) or A/B compartment prediction (Fortin and Hansen 2015). Our differential methylation workflow presented here demonstrates how to read in data, perform quality control

and filtering, normalisation and differential methylation testing. In addition we demonstrate analysis for differential variability, gene set testing and estimating cell type composition. One important aspect of exploring results of an analysis is visualisation and we also provide an example of generating region-level views of the data.

5 Software versions

```
sessionInfo()
```

```
## R version 4.4.0 RC (2024-04-16 r86468)
## Platform: x86_64-pc-linux-gnu
## Running under: Ubuntu 22.04.4 LTS
##
## Matrix products: default
## BLAS:   /home/biocbuild/bbs-3.20-bioc/R/lib/libRblas.so
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.10.0
##
## locale:
##  [1] LC_CTYPE=en_US.UTF-8       LC_NUMERIC=C
##  [3] LC_TIME=en_GB              LC_COLLATE=C
##  [5] LC_MONETARY=en_US.UTF-8    LC_MESSAGES=en_US.UTF-8
##  [7] LC_PAPER=en_US.UTF-8       LC_NAME=C
##  [9] LC_ADDRESS=C               LC_TELEPHONE=C
## [11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C
##
## time zone: America/New_York
## tzcode source: system (glibc)
##
## attached base packages:
##  [1] grid      parallel  stats4    stats     graphics  grDevices u
##  [8] datasets  methods   base
##
## other attached packages:
##  [1] DMRcatedata_2.21.3
##  [2] ExperimentHub_2.11.3
##  [3] AnnotationHub_3.11.5
##  [4] BiocFileCache_2.11.2
##  [5] dbplyr_2.5.0
##  [6] stringr_1.5.1
##  [7] DMRcate_2.99.6
##  [8] Gviz_1.47.1
##  [9] minfiData_0.49.0
## [10] missMethyl_1.37.0
## [11] IlluminaHumanMethylationEPICanno.ilm10b4.hg19_0.6.0
## [12] RColorBrewer_1.1-3
## [13] IlluminaHumanMethylation450kmanifest_0.4.0
## [14] IlluminaHumanMethylation450kanno.ilmn12.hg19_0.6.1
## [15] minfi_1.49.1
## [16] bumphunter_1.45.1
## [17] locfit_1.5-9.9
## [18] iterators_1.0.14
## [19] foreach_1.5.2
## [20] Biostrings_2.71.6
## [21] XVector_0.43.1
## [22] SummarizedExperiment_1.33.3
## [23] Biobase_2.63.1
## [24] MatrixGenerics_1.15.1
## [25] matrixStats_1.3.0
## [26] GenomicRanges_1.55.4
## [27] GenomeInfoDb_1.39.14
## [28] IRanges_2.37.1
## [29] S4Vectors_0.41.7
## [30] BiocGenerics_0.49.1
## [31] limma_3.59.10
## [32] knitr_1.46
```

```
## [33] BiocStyle_2.31.0
##
## loaded via a namespace (and not attached):
##   [1] ProtGenerics_1.35.4       bitops_1.0-7
##   [3] httr_1.4.7                tools_4.4.0
##   [5] doRNG_1.8.6               backports_1.4.1
##   [7] utf8_1.2.4                R6_2.5.1
##   [9] HDF5Array_1.31.6          lazyeval_0.2.2
##  [11] rhdf5filters_1.15.5       permute_0.9-7
##  [13] withr_3.0.0               prettyunits_1.2.0
##  [15] gridExtra_2.3             base64_2.0.1
##  [17] preprocessCore_1.65.0     cli_3.6.2
##  [19] sass_0.4.9                readr_2.1.5
##  [21] genefilter_1.85.1         askpass_1.2.0
##  [23] Rsamtools_2.19.4          foreign_0.8-86
##  [25] siggenes_1.77.0           illuminaio_0.45.0
##  [27] R.utils_2.12.3            dichromat_2.0-0.1
##  [29] scrime_1.3.5              BSgenome_1.71.4
##  [31] readxl_1.4.3              rstudioapi_0.16.0
##  [33] RSQLite_2.3.6             generics_0.1.3
##  [35] BiocIO_1.13.1             gtools_3.9.5
##  [37] dplyr_1.1.4               GO.db_3.19.1
##  [39] Matrix_1.7-0              interp_1.1-6
##  [41] fansi_1.0.6               abind_1.4-5
##  [43] R.methodsS3_1.8.2         lifecycle_1.0.4
##  [45] yaml_2.3.8                edgeR_4.1.33
##  [47] rhdf5_2.47.7              SparseArray_1.3.7
##  [49] blob_1.2.4                crayon_1.5.2
##  [51] lattice_0.22-6            GenomicFeatures_1.55.4
##  [53] annotate_1.81.2           KEGGREST_1.43.1
##  [55] magick_2.8.3              pillar_1.9.0
##  [57] beanplot_1.3.1            rjson_0.2.21
##  [59] codetools_0.2-20          glue_1.7.0
##  [61] data.table_1.15.4         vctrs_0.6.5
##  [63] png_0.1-8                 cellranger_1.1.0
##  [65] gtable_0.3.5              cachem_1.0.8
##  [67] xfun_0.43                 S4Arrays_1.3.7
##  [69] mime_0.12                 survival_3.6-4
##  [71] tinytex_0.50              statmod_1.5.0
##  [73] nlme_3.1-164              bit64_4.0.5
##  [75] bsseq_1.39.0              progress_1.2.3
##  [77] filelock_1.0.3            bslib_0.7.0
##  [79] nor1mix_1.3-3             rpart_4.1.23
##  [81] colorspace_2.1-0          DBI_1.2.2
##  [83] Hmisc_5.1-2               nnet_7.3-19
##  [85] tidyselect_1.2.1          bit_4.0.5
##  [87] compiler_4.4.0            curl_5.2.1
##  [89] httr2_1.0.1               htmlTable_2.4.2
##  [91] BiasedUrn_2.0.11          xml2_1.3.6
##  [93] DelayedArray_0.29.9       bookdown_0.39
##  [95] rtracklayer_1.63.3        checkmate_2.3.1
##  [97] scales_1.3.0              quadprog_1.5-8
##  [99] rappdirs_0.3.3            digest_0.6.35
## [101] rmarkdown_2.26            GEOquery_2.71.0
## [103] htmltools_0.5.8.1         pkgconfig_2.0.3
## [105] jpeg_0.1-10               base64enc_0.1-3
## [107] sparseMatrixStats_1.15.1  highr_0.10
```

```
## [109] fastmap_1.1.1             ensembldb_2.27.1
## [111] rlang_1.1.3               htmlwidgets_1.6.4
## [113] UCSC.utils_0.99.7         DelayedMatrixStats_1.25.4
## [115] jquerylib_0.1.4           jsonlite_1.8.8
## [117] BiocParallel_1.37.1       mclust_6.1.1
## [119] R.oo_1.26.0               VariantAnnotation_1.49.7
## [121] RCurl_1.98-1.14           magrittr_2.0.3
## [123] Formula_1.2-5             GenomeInfoDbData_1.2.12
## [125] Rhdf5lib_1.25.3           munsell_0.5.1
## [127] Rcpp_1.0.12               stringi_1.8.3
## [129] zlibbioc_1.49.3           MASS_7.3-60.2
## [131] plyr_1.8.9                org.Hs.eg.db_3.19.1
## [133] deldir_2.0-4              splines_4.4.0
## [135] multtest_2.59.0           hms_1.1.3
## [137] rngtools_1.5.2            biomaRt_2.59.1
## [139] BiocVersion_3.19.1        XML_3.99-0.16.1
## [141] evaluate_0.23             latticeExtra_0.6-30
## [143] biovizBase_1.51.0         BiocManager_1.30.22
## [145] tzdb_0.4.0                tidyr_1.3.1
## [147] openssl_2.1.2             purrr_1.0.2
## [149] reshape_0.8.9             ggplot2_3.5.1
## [151] xtable_1.8-4              restfulr_0.0.15
## [153] AnnotationFilter_1.27.0   tibble_3.2.1
## [155] memoise_2.0.1             AnnotationDbi_1.65.2
## [157] GenomicAlignments 1.39.5  cluster 2.1.6
```

6 Author contributions

JM and BP designed the content and wrote the paper. AO oversaw the project and contributed to the writing and editing of the paper.

7 Competing interests

No competing interests were disclosed.

8 Grant information

AO was supported by an NHMRC Career Development Fellowship APP1051481.

References

Aryee, Martin J, Andrew E Jaffe, Hector Corrada-Bravo, Christine Ladd-Acosta, Andrew P Feinberg, Kasper D Hansen, and Rafael a Irizarry. 2014. “Minfi: a flexible and comprehensive Bioconductor package for the analysis of Infinium DNA methylation microarrays.” *Bioinformatics (Oxford, England)* 30 (10): 1363–9. https://doi.org/10.1093/bioinformatics/btu049 (https://doi.org/10.1093/bioinformatics/btu049).

Aryee, Martin J, Zhijin Wu, Christine Ladd-Acosta, Brian Herb, Andrew P Feinberg, Srinivasan Yegnasubramanian, and Rafael a Irizarry. 2011. “Accurate genome-scale percentage DNA methylation estimates from microarray data.” *Biostatistics (Oxford, England)* 12 (2): 197–210. https://doi.org/10.1093/biostatistics/kxq055 (https://doi.org/10.1093/biostatistics/kxq055).

Benjamini, Y, and Y Hochberg. 1995. “Controlling the false discovery rate: a practical and powerful approach to multiple testing.” *Journal of the Royal Statistical Society: Series B* 57: 289–300.

Bibikova, Marina, Bret Barnes, Chan Tsan, Vincent Ho, Brandy Klotzle, Jennie M Le, David Delano, et al. 2011. “High density DNA methylation array with single CpG site resolution.” *Genomics* 98 (4): 288–95. https://doi.org/10.1016/j.ygeno.2011.07.007 (https://doi.org/10.1016/j.ygeno.2011.07.007).

Bibikova, Marina, Jennie Le, Bret Barnes, Shadi Saedinia-Melnyk, Lixin Zhou, Richard Shen, and Kevin L Gunderson. 2009. “Genome-wide DNA methylation profiling using Infinium assay.” *Epigenomics* 1 (1): 177–200. https://doi.org/10.2217/epi.09.14 (https://doi.org/10.2217/epi.09.14).

Bird, Adrian. 2002. “DNA methylation patterns and epigenetic memory.” *Genes & Development* 16 (1): 6–21. https://doi.org/10.1101/gad.947102 (https://doi.org/10.1101/gad.947102).

Chen, Yi-an, Mathieu Lemire, Sanaa Choufani, Darci T Butcher, Daria Grafodatskaya, Brent W Zanke, Steven Gallinger, Thomas J Hudson, and Rosanna Weksberg. 2013. “Discovery of cross-reactive probes and polymorphic CpGs in the Illumina Infinium HumanMethylation450 microarray.” *Epigenetics : Official Journal of the DNA Methylation Society* 8 (2): 203–9. https://doi.org/10.4161/epi.23470 (https://doi.org/10.4161/epi.23470).

Cruickshank, Mark N, Alicia Oshlack, Christiane Theda, Peter G Davis, David Martino, Penelope Sheehan, Yun Dai, Richard Saffery, Lex W Doyle, and Jeffrey M Craig. 2013. “Analysis of epigenetic changes in survivors of preterm birth reveals the effect of gestational age and evidence for a long term legacy.” *Genome Medicine* 5 (10): 96. https://doi.org/10.1186/gm500 (https://doi.org/10.1186/gm500).

Davis, Sean, Pan Du, Sven Bilke, Tim Triche, Jr., and Moiz Bootwalla. 2015. *Methylumi: Handle Illumina Methylation Data*.

Du, Pan, Xiao Zhang, Chiang-Ching Huang, Nadereh Jafari, Warren a Kibbe, Lifang Hou, and Simon M Lin. 2010. “Comparison of Beta-value and M-value methods for quantifying methylation levels by microarray analysis.” *BMC Bioinformatics* 11 (1): 587. https://doi.org/10.1186/1471-2105-11-587 (https://doi.org/10.1186/1471-2105-11-587).

Fortin, JP, and KD Hansen. 2015. “Reconstructing A/B compartments as revealed by Hi-C using long-range correlations in epigenetic data.” *Genome Biology* 16 (1): 180. https://doi.org/10.1186/s13059-015-0741-y (https://doi.org/10.1186/s13059-015-0741-y).

Fortin, JP, Aurélie Labbe, Mathieu Lemire, and BW Zanke. 2014. “Functional normalization of 450k methylation array data improves replication in large cancer studies.” *Genome Biology* 15 (503): 1–17.

Hansen, Kasper Daniel, Winston Timp, Héctor Corrada Bravo, Sarven Sabunciyan, Benjamin Langmead, Oliver G McDonald, Bo Wen, et al. 2011. “Increased methylation variation in epigenetic domains across cancer types.” *Nature Genetics* 43 (8): 768–75. https://doi.org/10.1038/ng.865 (https://doi.org/10.1038/ng.865).

Heyn, Holger, Ning Li, HJ Humberto J Ferreira, Sebastian Moran, David G Pisano, Antonio Gomez, and Javier Diez. 2012. “Distinct DNA methylomes of newborns and centenarians.” *Proceedings of the National Academy of Sciences of the United States of America* 109 (26): 10522–7. https://doi.org/10.1073/pnas.1120658109/-/DCSupplemental.www.pnas.org/cgi/doi/10.1073/pnas.1120658109 (https://doi.org/10.1073/pnas.1120658109/-/DCSupplemental.www.pnas.org/cgi/doi/10.1073/pnas.1120658109).

Hicks, Stephanie C, and Rafael A Irizarry. 2015. “Quantro: A Data-Driven Approach to Guide the Choice of an Appropriate Normalization Method.” *Genome Biology* 16 (1): 1.

Houseman, Eugene Andres, William P Accomando, Devin C Koestler, Brock C Christensen, Carmen J Marsit, Heather H Nelson, John K Wiencke, and Karl T Kelsey. 2012. “DNA methylation arrays as surrogate measures of cell mixture distribution.” *BMC Bioinformatics* 13 (1): 86. https://doi.org/10.1186/1471-2105-13-86 (https://doi.org/10.1186/1471-2105-13-86).

Huber, W., Carey, V. J., Gentleman, R., Anders, et al. 2015. “Orchestrating High-Throughput Genomic Analysis with Bioconductor.” *Nature Methods* 12 (2): 115–21. http://www.nature.com/nmeth/journal/v12/n2/full/nmeth.3252.html (http://www.nature.com/nmeth/journal/v12/n2/full/nmeth.3252.html).

Jaffe, Andrew E, and Rafael a Irizarry. 2014. “Accounting for cellular heterogeneity is critical in epigenome-wide association studies.” *Genome Biology* 15 (2): R31. https://doi.org/10.1186/gb-2014-15-2-r31 (https://doi.org/10.1186/gb-2014-15-2-r31).

Jaffe, Andrew E, Peter Murakami, Hwajin Lee, Jeffrey T Leek, M Daniele Fallin, Andrew P Feinberg, and Rafael a Irizarry. 2012. “Bump hunting to identify differentially methylated regions in epigenetic epidemiology studies.” *International Journal of Epidemiology* 41 (1): 200–209. https://doi.org/10.1093/ije/dyr238 (https://doi.org/10.1093/ije/dyr238).

Laird, Peter W. 2003. “The power and the promise of DNA methylation markers.” *Nature Reviews. Cancer* 3 (4): 253–66. https://doi.org/10.1038/nrc1045 (https://doi.org/10.1038/nrc1045).

Leek, Jeffrey T, W Evan Johnson, Hilary S Parker, Andrew E Jaffe, and John D Storey. 2012. “The sva package for removing batch effects and other unwanted variation in high-throughput experiments.” *Bioinformatics (Oxford, England)* 28 (6): 882–3. https://doi.org/10.1093/bioinformatics/bts034 (https://doi.org/10.1093/bioinformatics/bts034).

Lonnstedt, I, and T Speed. 2002. “Replicated Microarray Data.” *Statistica Sinica* 12: 31–46.

Maksimovic, Jovana, Johann A Gagnon-Bartsch, Terence P Speed, and Alicia Oshlack. 2015. “Removing unwanted variation in a differential methylation analysis of Illumina HumanMethylation450 array data.” *Nucleic Acids Research*, May, gkv526. https://doi.org/10.1093/nar/gkv526 (https://doi.org/10.1093/nar/gkv526).

Maksimovic, Jovana, Lavinia Gordon, and Alicia Oshlack. 2012. “SWAN: Subset-quantile within array normalization for illumina infinium HumanMethylation450 BeadChips.” *Genome Biology* 13 (6): R44. https://doi.org/10.1186/gb-2012-13-6-r44 (https://doi.org/10.1186/gb-2012-13-6-r44).

Mancuso, Francesco M, Magda Montfort, Anna Carreras, Andreu Alibés, and Guglielmo Roma. 2011. “HumMeth27QCReport: an R package for quality control and primary analysis of Illumina Infinium methylation data.” *BMC Research Notes* 4 (January): 546. https://doi.org/10.1186/1756-0500-4-546 (https://doi.org/10.1186/1756-0500-4-546).

Morris, Tiffany J, Lee M Butcher, Andrew Feber, Andrew E Teschendorff, Ankur R Chakravarthy, Tomasz K Wojdacz, and Stephan Beck. 2014. “ChAMP: 450k Chip Analysis

Methylation Pipeline." *Bioinformatics (Oxford, England)* 30 (3): 428–30. https://doi.org/10.1093/bioinformatics/btt684 (https://doi.org/10.1093/bioinformatics/btt684).

Peters, Timothy J, Michael J Buckley, Aaron L Statham, Ruth Pidsley, Katherine Samaras, Reginald V Lord, Susan J Clark, and Peter L Molloy. 2015. "De novo identification of differentially methylated regions in the human genome." *Epigenetics & Chromatin* 8 (1): 6. https://doi.org/10.1186/1756-8935-8-6 (https://doi.org/10.1186/1756-8935-8-6).

Phipson, Belinda, Jovana Maksimovic, and Alicia Oshlack. 2016. "missMethyl: an R package for analyzing data from Illumina's HumanMethylation450 platform." *Bioinformatics (Oxford, England)* 32 (2): 286–88. https://doi.org/10.1093/bioinformatics/btv560 (https://doi.org/10.1093/bioinformatics/btv560).

Phipson, Belinda, and Alicia Oshlack. 2014. "DiffVar: a new method for detecting differential variability with application to methylation in cancer and aging." *Genome Biology* 15 (9): 465. https://doi.org/10.1186/s13059-014-0465-4 (https://doi.org/10.1186/s13059-014-0465-4).

Pidsley, Ruth, Chloe C Y Wong, Manuela Volta, Katie Lunnon, Jonathan Mill, and Leonard C Schalkwyk. 2013. "A data-driven approach to preprocessing Illumina 450K methylation array data." *BMC Genomics* 14 (1): 293. https://doi.org/10.1186/1471-2164-14-293 (https://doi.org/10.1186/1471-2164-14-293).

R Core Team. 2014. "R: A language and environment for statistical computing." Vienna, Austria: R Foundation for Statistical Computing. http://www.r-project.org/ (http://www.r-project.org/).

Ritchie, M. E., B. Phipson, D. Wu, Y. Hu, C. W. Law, W. Shi, and G. K. Smyth. 2015. "limma powers differential expression analyses for RNA-sequencing and microarray studies." *Nucleic Acids Research*, January, gkv007. https://doi.org/10.1093/nar/gkv007 (https://doi.org/10.1093/nar/gkv007).

Smith, Mike L, Keith A. Baggerly, Henrik Bengtsson, Matthew E. Ritchie, Kasper D. Hansen, Mike L Smith, Keith A. Baggerly, Henrik Bengtsson, Matthew E. Ritchie, and Kasper D. Hansen. 2013. "illuminaio: An open source IDAT parsing tool for Illumina microarrays." *F1000Research* 2 (December). https://doi.org/10.12688/f1000research.2-264.v1 (https://doi.org/10.12688/f1000research.2-264.v1).

Smyth, G K. 2004. “Linear models and empirical Bayes methods for assessing differential expression in microarray experiments.” *Statistical Applications in Genetics and Molecular Biology* 3 (1): Article~3.

Sun, Zhifu, High Seng Chai, Yanhong Wu, Wendy M White, Krishna V Donkena, Christopher J Klein, Vesna D Garovic, Terry M Therneau, and Jean-Pierre a Kocher. 2011. “Batch effect correction for genome-wide methylation data with Illumina Infinium platform.” *BMC Medical Genomics* 4 (January): 84. https://doi.org/10.1186/1755-8794-4-84 (https://doi.org/10.1186/1755-8794-4-84).

Teschendorff, Andrew E, Francesco Marabita, Matthias Lechner, Thomas Bartlett, Jesper Tegner, David Gomez-Cabrero, and Stephan Beck. 2013. “A beta-mixture quantile normalization method for correcting probe design bias in Illumina Infinium 450 k DNA methylation data.” *Bioinformatics (Oxford, England)* 29 (2): 189–96. https://doi.org/10.1093/bioinformatics/bts680 (https://doi.org/10.1093/bioinformatics/bts680).

Teschendorff, Andrew E, Joanna Zhuang, and Martin Widschwendter. 2011. “Independent surrogate variable analysis to deconvolve confounding factors in large-scale microarray profiling studies.” *Bioinformatics (Oxford, England)* 27 (11): 1496–1505. https://doi.org/10.1093/bioinformatics/btr171 (https://doi.org/10.1093/bioinformatics/btr171).

Touleimat, Nizar, and Jörg Tost. 2012. “Complete pipeline for Infinium Human Methylation 450K BeadChip data processing using subset quantile normalization for accurate DNA methylation estimation.” *Epigenomics* 4 (3): 325–41. https://doi.org/10.2217/epi.12.21 (https://doi.org/10.2217/epi.12.21).

Triche, Timothy J, Daniel J Weisenberger, David Van Den Berg, Peter W Laird, and Kimberly D Siegmund. 2013. “Low-level processing of Illumina Infinium DNA Methylation BeadArrays.” *Nucleic Acids Research* 41 (7): e90. https://doi.org/10.1093/nar/gkt090 (https://doi.org/10.1093/nar/gkt090).

Wang, Dong, Yuannv Zhang, Yan Huang, Pengfei Li, Mingyue Wang, Ruihong Wu, Lixin Cheng, et al. 2012. “Comparison of different normalization assumptions for analyses of DNA methylation data from the cancer genome.” *Gene* 506 (1): 36–42. https://doi.org/10.1016/j.gene.2012.06.075 (https://doi.org/10.1016/j.gene.2012.06.075).

Wu, Hao, Brian Caffo, Harris A Jaffee, Rafael A Irizarry, and Andrew P Feinberg. 2010. “Redefining CpG islands using hidden Markov models.” *Biostatistics (Oxford, England)* 11 (3): 499–514. https://doi.org/10.1093/biostatistics/kxq005 (https://doi.org/10.1093/biostatistics/kxq005).

Wu, Michael C, Bonnie R Joubert, Pei-fen Kuan, Siri E Håberg, Wenche Nystad, Shyamal D Peddada, and Stephanie J London. 2014. “A systematic assessment of normalization approaches for the Infinium 450K methylation platform.” *Epigenetics : Official Journal of the DNA Methylation Society* 9 (2): 318–29. https://doi.org/10.4161/epi.27119 (https://doi.org/10.4161/epi.27119).

Zhang, Yuxia, Jovana Maksimovic, Gaetano Naselli, Junyan Qian, Michael Chopin, Marnie E Blewitt, Alicia Oshlack, and Leonard C Harrison. 2013. “Genome-wide DNA methylation analysis identifies hypomethylated genes regulated by FOXP3 in human regulatory T cells.” *Blood* 122 (16): 2823–36. https://doi.org/10.1182/blood-2013-02-481788 (https://doi.org/10.1182/blood-2013-02-481788).